

Gabriel Garcia Almeida
168857

Implementação de um sistema operacional simplificado

São José dos Campos - Brasil
Setembro de 2026

Gabriel Garcia Almeida
168857

Implementação de um sistema operacional simplificado

Relatório apresentado à Universidade Federal de São Paulo como parte dos requisitos para aprovação na disciplina de Laboratório de Sistemas Computacionais: Sistemas Operacionais.

Docente: Prof. Dr. Tiago de Oliveira

Universidade Federal de São Paulo - UNIFESP

Instituto de Ciência e Tecnologia - Campus São José dos Campos

São José dos Campos - Brasil

Setembro de 2026

Resumo

Esse documento tem o propósito de apresentar o planejamento e a implementação de um sistema operacional simplificado utilizando aspectos de software e de hardware. Para a compilação do código foi utilizado um compilador escrito na linguagem C, que traduz o código-fonte escrito em C- para código intermediário, assembly e código binário. O código binário gerado é enviado para um processador MIPS monociclo, escrito em Verilog e implementado em kit FPGA. O sistema operacional desenvolvido gerencia a execução de dez programas desenvolvidos, podendo ser executados de maneira sequencial ou concorrente.

Palavras-chave: sistema operacional, compilador, processador MIPS, kit FPGA.

Lista de ilustrações

Figura 1 – Estados assumidos por um processo	9
Figura 2 – Caminho de dados	20
Figura 3 – Caminho de dados - Formato 1 (add, sub, mul, div, and, or, xnor) . . .	22
Figura 4 – Caminho de dados - Formato 3 desvio tomado (beq, bne, blt, bgt) . . .	22
Figura 5 – Caminho de dados - Formato 7 (pop)	23
Figura 6 – Caminho de dados - Formato 7 (pop)	26

Lista de tabelas

Tabela 1 – Registradores de uso específico	14
Tabela 2 – Lista numerada das Instruções	15
Tabela 3 – Classificação das Instruções	15
Tabela 4 – Instruções utilizadas e operações realizadas separadas por formato (Parte 1)	17
Tabela 5 – Instruções utilizadas e operações realizadas separadas por formato (Parte 2)	18
Tabela 6 – Todos os formatos de instruções	19
Tabela 7 – Classificação das Instruções por Formato	19
Tabela 8 – Sinais de controle e suas funções	21
Tabela 9 – Entradas e saídas do módulo da ULA	24
Tabela 10 – Operações realizadas pela ULA conforme o SinalULA	24
Tabela 11 – Entradas e saídas do módulo do banco de registradores	25
Tabela 12 – Entradas e saídas do módulo da memória principal	27
Tabela 13 – Entradas e saídas do módulo da memória de instruções	28
Tabela 14 – Entradas e saídas do módulo PC	29
Tabela 15 – Entradas e saídas da unidade de controle	30
Tabela 16 – Entradas e saídas do módulo de entrada e saída	31
Tabela 17 – Entradas e saídas da integração final	32
Tabela 18 – Tipos de quádrupla gerados pelo compilador	71
Tabela 19 – Mensagens do LCD	78

Sumário

1	INTRODUÇÃO	7
2	OBJETIVOS	8
2.1	Geral	8
2.2	Específicos	8
3	FUNDAMENTAÇÃO TEÓRICA	9
3.1	Gerenciamento de processos	9
3.1.1	Algoritmo Round-Robin	10
3.2	Gerenciamento de memória	10
3.3	Gerenciamento de sistemas de arquivos	11
3.3.1	Formas de armazenar os arquivos	11
3.4	Gerenciamento de dispositivos de E/S	12
3.4.1	Entrada e saída mapeadas na memória	12
3.4.2	Acesso direto à memória (DMA)	12
3.5	FPGA	13
3.6	Linguagem Verilog HDL	13
4	DESENVOLVIMENTO DO PROCESSADOR	14
4.1	Características do projeto	14
4.2	Conjunto de instruções	14
4.3	Formato de instruções	18
4.4	Caminho de dados	19
4.5	Descrição dos módulos	23
4.5.1	ULA	23
4.5.2	Banco de registradores	25
4.5.3	Integração do banco de registrador e ULA	26
4.5.4	Memória principal	27
4.5.5	Memória de instruções	27
4.5.6	PC	28
4.5.7	Unidade de controle	29
4.5.8	Módulo de entrada e saída	30
4.5.9	Integração final	31
4.6	Implementações	32
4.6.1	ULA	32
4.6.2	Banco de registradores	34

4.6.3	Integração ULA com banco de registradores	35
4.6.4	Memória principal	37
4.6.5	Memória instruções	38
4.6.6	PC	39
4.6.7	Unidade de controle	40
4.6.8	Módulo de entrada e saída	53
4.6.9	Integração final	56
5	DESENVOLVIMENTO DO COMPILADOR	61
5.1	Fase de Análise	61
5.1.1	Modelagem	61
5.1.1.1	Diagrama de Blocos (SysML)	61
5.1.1.2	Diagramas de Atividades (SysML)	61
5.1.1.3	Outros: a Árvore Sintática Abstrata	62
5.1.2	Análise Léxica	62
5.1.3	Análise Sintática	64
5.1.4	Análise Semântica	66
5.2	Fase de Síntese	69
5.2.1	Modelagem	69
5.2.1.1	Diagrama de Blocos (SysML)	69
5.2.1.2	Diagramas de Atividades (SysML)	69
5.2.2	Geração de Código Intermediário	69
5.2.3	Geração de Código Assembly	72
5.2.4	Geração de Código Executável	74
5.2.5	Gerenciamento de Memória	76
6	DESENVOLVIMENTO DO SISTEMA OPERACIONAL	78
6.1	Gerenciamento de processos	78
6.2	Gerenciamento de entrada e saída	78
6.2.1	Interface	78
7	CONCLUSÃO	80
7.1	Dificuldades Encontradas	80
7.2	Destaques	81
7.3	Trabalhos Futuros	81
	REFERÊNCIAS	83

1 Introdução

Conforme o tempo passa, mais a tecnologia utilizadas nos computadores aumenta e cada vez mais os usuários desejam executar atividades de maneira concorrente: utilizar um determinado programa, enquanto deixa um navegador em aberto e uma música tocando. Para que isso seja possível, é necessário que o computador possua uma maneira de lidar com vários programas sendo executados de forma concorrente, de modo que a execução pareça ser simultânea para o usuário.

Além disso, da perspectiva de um programador, é essencial que haja uma maneira de se preocupar apenas com os detalhes de alto nível da aplicação, sem se preocupar em como exatamente cada um dos recursos é utilizado pela máquina. Isso permite que o programador se concentre em desenvolver a aplicação, sem se preocupar com os detalhes de implementação do hardware.

De modo a lidar com essas necessidades, surgiu o conceito de sistema operacional, que é um software responsável por gerenciar os recursos de hardware e software do computador, que, dentre outras coisas, permite com que vários programas sejam executados de maneira concorrente e que o programador possa se concentrar em desenvolver a aplicação sem se preocupar com os detalhes de implementação do hardware.

Entender o funcionamento de um sistema operacional é uma parte fundamental para a formação de um engenheiro da computação, que deve ser capaz de compreender como o sistema operacional gerencia os recursos do computador e, mais do que isso, de desenvolver um sistema operacional simplificado por conta própria.

Além do mais, para validar o funcionamento do sistema operacional desenvolvido, serão utilizadas ferramentas de projetos anteriores: um processador MIPS escrito em Verilog implementado em FPGA e um compilador da linguagem C- para esse processador em questão.

Quanto à organização do texto, inicialmente são apresentados os objetivos do projeto. Em seguida, é retomado o processador que serve de alvo para o compilador, com a sua fundamentação teórica, o seu desenvolvimento e os resultados obtidos. Após isso será apresentado o compilador desenvolvido anteriormente, a fase de análise e a fase de síntese do compilador e exemplos de códigos compilados. Em sequência é apresentado.

2 Objetivos

2.1 Geral

O objetivo geral é desenvolver um sistema operacional simplificado capaz de lidar com a execução de um único programa ou múltiplos programas de maneira concorrente. Além disso, validar o funcionamento do sistema operacional utilizando um compilador para C- e usando o código binário gerado em um processador RISC implementado na linguagem Verilog utilizando o software Quartus e implementá-lo no kit FPGA.

2.2 Específicos

1. Desenvolver 10 códigos em C-, compilar e testar no kit FPGA, para que possam ser utilizados como programas de teste do sistema operacional;
2. Fazer o planejamento da implementação de um sistema operacional simplificado;
3. Implementar o módulo de interrupção, responsável por definir o quantum de cada programa e por alternar entre os programas em execução;
4. Implementar o controle de acesso dinâmico à memória principal e à memória de instruções de acordo com o programa que é executado;
5. Desenvolver o código do sistema operacional em C- para fazer o escalonamento entre os processos utilizando Round Robin e implementar a interface;
6. Compilar o código do sistema operacional junto com os outros 10 códigos em C- e testar no kit FPGA, para verificar se o sistema operacional é capaz de gerenciar a execução dos programas corretamente;

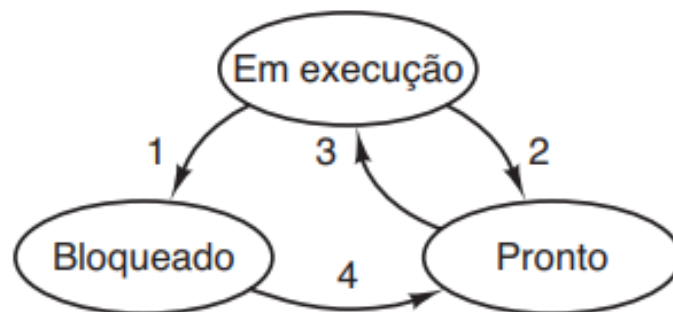
3 Fundamentação Teórica

3.1 Gerenciamento de processos

Primeiramente, Um processo é um programa computacional que está em execução (TANENBAUM; BOS, 2016). Sendo assim, como o projeto desenvolvido possui um único processador, apenas um processo pode ser executado por vez. Quando há um único programa na memória de instruções, não há nenhum problema, porém, quando há múltiplos programas na memória de instruções, ou seja, quando é um sistema multiprogramado, se torna um desafio, pois apenas um programa pode ser executado por vez. Isso gera a necessidade de fazer um controle de qual processo será executado de cada vez, uma responsabilidade atribuída ao sistema operacional. (TANENBAUM; BOS, 2016).

Para lidar com isso, existe uma parte do sistema operacional chamada escalonador, que junto à um algoritmo de escalonamento, decide qual deve ser o próximo programa a ser executado (TANENBAUM; BOS, 2016). Esse processo depende, dentre outras coisas, do estado de cada um dos processos, que varia ao longo da sua execução. Os principais estados dos processos podem ser observados na Figura 1.

Figura 1 – Estados assumidos por um processo



1. O processo é bloqueado aguardando uma entrada
2. O escalonador seleciona outro processo
3. O escalonador seleciona esse processo
4. A entrada torna-se disponível

Fonte: Sistemas Operacionais Modernos (TANENBAUM; BOS, 2016).

Um processo está: em execução quando ele está utilizando a CPU para executar as instruções do programa; pronto quando pode começar a ser executado a qualquer momento, porém está esperando outro termina a sua execução; bloqueado quando está esperando por algum evento externo, como a entrada de dados do usuário ou a finalização de uma

operação de entrada e saída (TANENBAUM; BOS, 2016).

Os algoritmos de escalonamento se dividem em dois grupos principais: os não preemptivos e os preemptivos. O primeiro grupo escolhe um processo e deixa com que ele execute até chegar ao final da sua execução, ou até que ele seja bloqueado. O segundo grupo escolhe um processo e permite que ele execute por um determinado período de tempo, chamado quantum, e depois o interrompe para que outro processo possa ser executado (TANENBAUM; BOS, 2016).

3.1.1 Algoritmo Round-Robin

Round-Robin é um dos algoritmos de escalonamento preemptivo, ou seja, ele troca o processo que está execução a cada um determinado tempo. Além disso, utiliza uma lista, na qual o primeiro processo é o próximo a ser executado e todo processo interrompido é colocado ao final da lista (TANENBAUM; BOS, 2016).

3.2 Gerenciamento de memória

Idealmente, haveria uma memória ilimitada, rápida e não volátil, porém, na realidade, não existe esse tipo de tecnologia. Desse modo, para lidar com essa questão, é utilizada a hierarquia de memória: pouca memória de alta velocidade volátil (registradores e cache), média quantidade de memória de velocidade média volátil (memória principal) e grande quantidade de memória de baixa velocidade não volátil(memória secundária) (TANENBAUM; BOS, 2016).

Como a memória é um recurso limitado, cabe ao sistema operacional gerenciar o acesso à memória entre os processos, sem que um comprometa os dados dos outros. Isso é feito em todos os níveis da memória, desde os registradores até a memória secundária.

Em nível dos registradores, é preciso que o sistema operacional preserve o conteúdo deles quando um processo é interrompido e retorne o conteúdo correto quando o processo for retomado. Em nível dos demais níveis da memória, é preciso garantir que um processo não acesse indevidamente a memória pertencente a outro processo por meio do conceito de espaço de endereçamento.

O espaço de endereçamento é uma espécie de abstração da memória que define quais são as posições que um determinado processo pode acessar na memória (TANENBAUM; BOS, 2016). Isso é utilizado para fazer com que o endereço lógico seja distinto do endereço físico, por exemplo, o endereço 47 utilizado para acessar a uma determinada instrução, não necessariamente estará na posição 47 da memória. Como todo programa pressupõe que ele próprio começa na linha zero, o gerenciamento da memória é uma parte fundamental para o sistema operacional coordenar a execução dos processos para que acessem o conteúdo

que de fato precisam.

A partir do ponto que nem todos os programas caibam na memória principal ao mesmo tempo, pode ser utilizada a estratégia de *swapping*, que retira um programa da memória principal e o coloca na memória secundária, liberando espaço para outro programa. Quando o programa que foi retirado da memória principal precisa ser executado novamente, ele é colocado de volta na memória principal, substituindo outro programa que não está sendo utilizado no momento (TANENBAUM; BOS, 2016).

Outras estratégias de gerenciamento de memória podem ser utilizadas, como a paginação e a segmentação. A paginação consiste em dividir a memória em blocos de tamanho fixo, chamados páginas, e alocar as páginas de acordo com a necessidade dos processos. A segmentação, por outro lado, divide a memória em segmentos de tamanho variável, que podem ser alocados de acordo com as necessidades específicas dos processos. Essas abordagens permitem com que não seja necessário mover o programa como um todo para dentro ou para fora da memória principal (TANENBAUM; BOS, 2016).

3.3 Gerenciamento de sistemas de arquivos

O sistema operacional também é responsável por gerenciar o acesso aos sistemas de arquivos, que são responsáveis por armazenar e organizar os dados em dispositivos de armazenamento, como discos rígidos e unidades de estado sólido. Isso inclui a criação, leitura, escrita e exclusão de arquivos, bem como a organização dos arquivos em diretórios e subdiretórios (TANENBAUM; BOS, 2016).

3.3.1 Formas de armazenar os arquivos

Existem várias abordagens utilizadas para armazenar os arquivos na memória, abaixo são apresentadas algumas delas (TANENBAUM; BOS, 2016):

- Alocação contígua - Os arquivos são separados em blocos contíguos na memória, ou seja, os blocos de um arquivo estão localizados em posições adjacentes. Isso permite um acesso rápido aos arquivos, porém pode levar a fragmentação da memória e dificuldade em encontrar espaço contíguo para novos arquivos.
- Alocação por lista encadeada - Cada arquivo é armazenado em blocos que podem estar espalhados pela memória, e cada bloco contém um ponteiro para o próximo bloco do arquivo. Isso permite uma utilização mais eficiente da memória, mas pode levar a um acesso mais lento aos arquivos, pois é necessário seguir os ponteiros para acessar todos os blocos de um arquivo.

- Alocação com lista encadeata e tabela - Cada arquivo é armazenado em blocos que podem estar espalhados pela memória, e uma tabela de alocação é utilizada para mapear os blocos de cada arquivo. Isso permite uma utilização eficiente da memória e um acesso rápido aos arquivos, mas pode levar a uma sobrecarga de gerenciamento da tabela de alocação.
- *I-nodes* - Cada arquivo é representado por um *i-node*, que contém informações sobre o arquivo, como seu tamanho, permissões e localização dos blocos de dados. Os *i-nodes* são armazenados em uma tabela separada da memória, permitindo uma utilização eficiente da memória e um acesso rápido aos arquivos.

3.4 Gerenciamento de dispositivos de E/S

Os processos são capazes de interagir com o mundo externo por meio de dispositivos de entrada e saída, como teclados, mouses, monitores e impressoras. Todavia, esses dispositivos são limitados e precisam ser compartilhados entre todos os processos simultaneamente. Desse modo, cabe ao sistema operacional gerenciar o acesso a esses dispositivos, garantindo que todos os processos possam utilizá-los (TANENBAUM; BOS, 2016).

3.4.1 Entrada e saída mapeadas na memória

A comunicação entre a CPU e os controladores de dispositivos ocorre por meio de registradores de controle e buffers de dados, podendo ser estruturada de duas formas principais: via espaços de endereçamento separados com instruções de E/S dedicadas ou por E/S mapeada na memória, na qual os dispositivos compartilham o mesmo espaço de endereçamento da RAM. (TANENBAUM; BOS, 2016)

3.4.2 Acesso direto à memória (DMA)

O Acesso Direto à Memória (DMA) é uma técnica que libera a CPU do trabalho custoso de copiar dados palavra por palavra entre os dispositivos de E/S e a memória principal, delegando essa tarefa a um controlador dedicado que gerencia as transferências diretamente pelo barramento. Ao ser programado pela CPU com o endereço de memória, a contagem de bytes e a direção do fluxo, o controlador de DMA executa a transferência — que pode operar por "roubo de ciclo" (uma palavra por vez) ou no modo "surto" (blocos inteiros) — e emite uma interrupção ao final do processo. Embora exija mecanismos como o uso de buffers internos nos controladores de dispositivos (para validação de dados via checksum e prevenção de erros de temporização) e apresente desvantagens como o custo adicional de hardware ou o bloqueio temporário do barramento, o DMA simplifica

o software do sistema operacional e otimiza drasticamente o desempenho do sistema em operações intensivas de E/S ([TANENBAUM; BOS, 2016](#)).

3.5 FPGA

Primeiramente, FPGA significa “Field Programmable Gate Array”. Ele é um circuito integrado, no qual é possível modificar muitas das funcionalidades por meio de como as portas lógicas e os flip-flop se conectam ([Intel, 2024](#)). Ou seja, um componente que, de acordo com os sinais de entrada, permite definir quais devem ser os sinais de saída por meio da programação.

O kit FPGA é, como o nome diz, é um FPGA, porém conectado a uma série de botões, switches, Leds, displays e outros periféricos, que permitem com que os projetos possam ser testados de uma forma simples e prática.

3.6 Linguagem Verilog HDL

A linguagem Verilog foi desenvolvida pela companhia Gateway Design Automation em 1985. Posteriormente, quatro anos depois, essa companhia foi adquirida pela Cadence Design System, que por sua vez, tornou o Verilog domínio público em 1990. Além disso, a linguagem faz parte do padrão IEEE ([MIDORIKAWA, 2007](#)).

O padrão IEEE (Institute of Electrical and Electronic Engineers) é um conjunto de normas técnicas internacionais. Ele proporciona portabilidade, contribuindo para a interconexão de produtos desenvolvidos por empresas distintas ([MIDORIKAWA, 2007](#)).

“O Verilog oferece ao projetista os meios para descrever um sistema digital em vários níveis de abstração, e também suporta ferramentas de projeto para síntese lógica.” ([MIDORIKAWA, 2007](#)).

Sendo assim, diferente de um circuito esquemático, no qual é desenvolvido usando as portas lógicas e elementos de memória diretamente, o Verilog é responsável por simular o comportamento lógico de uma forma abstrata, sem precisar se importar com as implementações nos mínimos detalhes.

4 Desenvolvimento do processador

4.1 Características do projeto

Para o desenvolvimento do processador foi escolhido como referência o modelo MIPS, pois simplifica a implementação. Porém também foram feitas algumas modificações para atender melhor aos objetivos propostos, como instruções para controlar a pilha; switches; display; esperar; e parar o processamento. Além do mais, seguirá a organização monociclo por ser mais simples de compreender e implementar.

Além disso, o campo de Opcode será composto por 6 bits, de modo que possam haver até 64 instruções. Ademais, serão utilizadas instruções com 32 bits e um total de 64 registradores. Desse modo, serão precisos 6 bits para referenciar um registrador. Do total de registradores, haverão 3 que serão de uso específico e estão apresentados na [Tabela 1](#) abaixo.

Tabela 1 – Registradores de uso específico

Registrador	Função
RAL	Armazena o retorno da instrução <i>jal</i>
RZ	Armazena o valor zero
RPI	Controla a pilha

Fonte: o Autor.

Serão utilizados os modos de endereçamento: imediato, para receber valores diretamente; direto, para instruções load e store; registrador, para receber o operando de um registrador; indireto de registrador, para operações load e store; deslocamento, para operações load e store; e por pilha, para controlar a pilha.

4.2 Conjunto de instruções

A [Tabela 2](#) abaixo apresenta as instruções escolhidas e o seu respectivo Opcode. Nela, o Opcode recebe um valor em binário, começando do zero.

Tabela 2 – Lista numerada das Instruções

Opcode	Instrução	Opcode	Instrução
000000	halt	010100	movr
000001	nop	010101	loaddi
000010	jump	010110	loadr
000011	jal	010111	beq
000100	addr	011000	bne
000101	subr	011001	blt
000110	mulr	011010	bgt
000111	divr	011011	storedi
001000	and	011100	storer
001001	or	011101	movi
001010	xnor	011110	loadi
001011	addi	011111	in
001100	subi	100000	pop
001101	muli	100001	out
001110	divi	100010	jr
001111	andi	100011	storei
010000	ori	100100	push
010001	not	100101	blet
010010	sl	100110	lget
010011	sr		

Fonte: o Autor.

A partir das instruções, é possível classificá-las de acordo com [Tabela 3](#) abaixo, de modo a atender ao objetivo específico ??.

Tabela 3 – Classificação das Instruções

Categoria	Instruções
Operações aritméticas	<i>addr, addi, subr, subi, mulr, muli, divr, divi</i>
Operações lógicas	<i>and, andi, or, ori, not, xnor, sl, sr</i>
Desvios condicionais	<i>beq, bne, blt, bgt, blet, lget</i>
Desvios incondicionais	<i>jump, jr, jal</i>
Acesso à memória	<i>storei, storer, storedi, loadi, loadr, loaddi</i>
Movimentação de dados	<i>movr, movi</i>
Controle da pilha	<i>push, pop</i>
Acesso aos switches	<i>in</i>
Acesso ao display	<i>out</i>
Controle de execução	<i>nop, halt</i>

Fonte: o Autor.

A [Tabela 4](#) e a [Tabela 5](#) contêm a operação a ser realizada por cada uma das instruções apresentadas acima, além de separar as instruções em nove formatos. Nela,

“<=” indica uma atribuição; RD, RS e RT são registradores; IM indica imediato; END representa o endereço; MEM indica acesso à memória; PILHA indica acesso à pilha.

Ainda sobre a [Tabela 4](#) e a [Tabela 5](#), o símbolo “~” indica uma inversão bit a bit. O número, quando ele aparece, indica o número de bits. A expressão “+RPI” indica que primeiro incrementa RPI e depois usa o valor, enquanto “RPI-” indica que primeiro usa o valor e depois decrementa.

Tabela 4 – Instruções utilizadas e operações realizadas separadas por formato (Parte 1)

Instrução	Operação
Formato 1	
addr	$RD \leq RS + RT$
subr	$RD \leq RS - RT$
mulr	$RD \leq RS * RT$
divr	$RD \leq RS / RT$
and	$RD \leq RS \text{ and } RT$
or	$RD \leq RS \text{ or } RT$
xnor	$RD \leq RS \text{ xnor } RT$
Formato 2	
addi	$RD \leq RS + IM14$
subi	$RD \leq RS - IM14$
muli	$RD \leq RS * IM14$
divi	$RD \leq RS / IM14$
andi	$RD \leq RS \text{ and } IM14$
ori	$RD \leq RS \text{ or } IM14$
not	$RD \leq \sim RS$
sl	$RD \leq RS \text{ deslocado } IM14 \text{ esq}$
sr	$RD \leq RS \text{ deslocado } IM14 \text{ dir}$
movr	$RD \leq RS$
loadi	$RD \leq MEM(RS + END14)$
loadr	$RD \leq MEM(RS)$
Formato 3	
beq	if($RS == RT$) $PC \leq PC+1+END14$
bne	if($RS != RT$) $PC \leq PC+1+END14$
blt	if($RS < RT$) $PC \leq PC+1+END14$
bgt	if($RS > RT$) $PC \leq PC+1+END14$
blet	if($RS \leq RT$) $PC \leq PC+1+END14$
lget	if($RS \geq RT$) $PC \leq PC+1+END14$
storedi	$MEM(RS + END14) \leq RT$
storer	$MEM(RS) \leq RT$
Formato 4	
movi	$RD \leq IM20$
loadi	$RD \leq MEM(END20)$
in	$RD \leq SWITCHES$
pop	$RD \leq PILHA(RPI-)$
Formato 5	
out	$DISPLAY \leq RS$
jr	$PC \leq RS$

Fonte: o Autor.

Tabela 5 – Instruções utilizadas e operações realizadas separadas por formato (Parte 2)

Instrução	Operação
Formato 6	
storei	MEM(END20) $\leq$ RT
push	PILHA(+RPI) $\leq$ RT
Formato 7	
halt	Para o processador
nop	Aguarda um ciclo de clock
jump	PC $\leq$ END26
jal	RAL $\leq$ PC; PC $\leq$ END26

Fonte: o Autor.

Observe que há três instruções distintas para load e para store e elas mudam a forma como o endereço da memória é obtido: loadi e storei usam um imediato; loadr e storer usam o endereço contido em um registrador; loadd e stored usam o endereço contido em um registrador somado com um imediato.

Seguindo uma lógica parecida, há as funções de salto incondicional: jmp, realiza um salto para o endereço de um imediato; jr, para um endereço guardado em um registrador; e jal, funciona como um jmp, porém também armazena o valor do retorno em RAL.

Além do mais, repare que há muitas instruções que possuem duas versões: uma usando somente registradores e outra, na qual o nome da instrução termina com a letra “i”, utilizando registradores e imediato.

Além disso, há instruções especiais como: in e out, que recebem a entrada dos switches e enviam o display, respectivamente; nop e hlt, responsáveis, respectivamente, por aguardar um ciclo de clock e parar o processador.

Ademais, nota-se que dentro de cada um dos formatos são sempre utilizados os mesmos registradores de uso geral (RD, RS e RT) e imediatos do mesmo tipo e tamanho. Na próxima seção é apresentado detalhadamente cada um dos formatos e como serão organizadas as suas instruções.

4.3 Formato de instruções

Como dito anteriormente, foram feitas algumas adaptações no modelo MIPS, de modo que para o projeto são usados nove formatos de instrução ao invés de apenas três. Além do mais, como os formatos definidos dependem dos registradores e dos imediatos/endereços usados, eles são distintos dos apresentados na [Tabela 3](#).

Desse modo, a [Tabela 6](#) apresenta como serão preenchidos os bits para as instruções de cada um dos formatos. Os campos com o símbolo “-” indicam que ele não é usado.

Tabela 6 – Todos os formatos de instruções

Bits	31 - 26	25 - 20	19 - 14	13 - 8	7 - 0
Formato 1	OP	RD	RS	RT	-
Formato 2	OP	RD	RS	IM14/END14/-	
Formato 3	OP	RS	RT	END14/-	
Formato 4	OP	RD	IM20/END20/-		
Formato 5	OP	RS	-		
Formato 6	OP	RT	END20/-		
Formato 7	OP	END26/-			

Fonte: o Autor.

Conforme a [Tabela 4](#) e a [Tabela 5](#), as instruções que pertencem a cada um dos formatos estão apresentadas na [Tabela 7](#) abaixo:

Tabela 7 – Classificação das Instruções por Formato

Formato	Instruções
Formato 1	<i>add, sub, mul, div, and, or, xnor</i>
Formato 2	<i>addi, subi, muli, divi, andi, ori, not, sl, sr, mov, load, loadr</i>
Formato 3	<i>beq, bne, blt, bgt, blet, lget, storedi, storer</i>
Formato 4	<i>movi, loadi, in, pop</i>
Formato 5	<i>out, jr</i>
Formato 6	<i>storei, push</i>
Formato 7	<i>hlt, nop, jmp, jal</i>

Fonte: o Autor.

4.4 Caminho de dados

A partir de tudo o que foi definido nas seções anteriores, é possível desenvolver o circuito do processador. A função dele é apresentar qual deverá ser o caminho que os dados devem passar para realizar a operação de cada uma das instruções.

Como o processador desenvolvido foi baseado no MIPS, há muita similaridade no caminho de dados, porém foi preciso adicionar novos elementos para comportar todas as instruções e os dispositivos de entrada e saída. A [Figura 2](#) apresenta o caminho de dados para o processador que será desenvolvido.

Tabela 8 – Sinais de controle e suas funções

Nome	Função
HLT	Quando ativo, impede o PC de ser incrementado
SinalMuxEE	Determina o dado para escrita especial
SinalGEN	Estabelece os registradores e o imediato que serão usados
SinalMuxPC	Define qual será o próximo valor do PC
SinalMuxE	Determina o dado para escrita
SinalMES	Indica se serão enviados dados para o display ou lido dados dos switches
SinalMemória	Define quando será feita a escrita de dados na memória
SinalULA	Aponta a operação a ser feita pela ULA
RegULA	Indica se o dado da segunda entrada da ULA virá de um registrador ou imediato
SinalBanco	Estabelece se será feita uma escrita e/ou escrita especial
SD	Sinal de saída da ULA que indica se haverá desvio ou não
TravaPC	Sinal de saída do módulo de entrada e saída que impede o PC de incrementar até um botão do FPGA ser apertado

Fonte: o Autor.

Em sequência, as figuras: [Figura 3](#), [Figura 4](#) e [Figura 5](#) apresentam o caminho de dados executados por algumas instruções. Neles, estão em vermelho os caminhos de dados que são usados e em verde os sinais de controle utilizados.

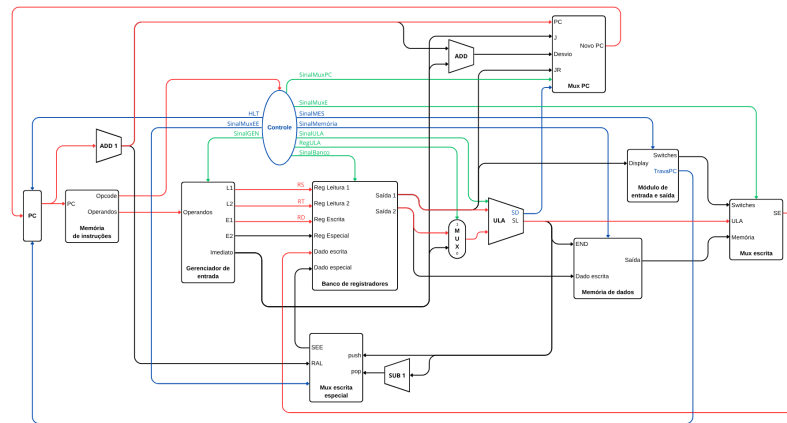
Observe que dentro de um mesmo formato é possível que haja um ou mais caminhos distintos para as instruções. Além disso, os demais caminhos podem ser obtidos tomando como referência os já apresentados, os sinais de controle e considerando a [Tabela 4](#) e [Tabela 5](#).

O primeiro caminho de dados, apresentado pela [Figura 3](#), mostra o caminho de dados das instruções do formato 1. Nele, assim como em todas as instruções, é lido o valor da memória de instruções apontada pelo PC e, em seguida, são determinados os registradores a serem usados.

Nesse caso, serão lidos os valores de dois registradores, RS e RT, e serão escritos dados no registrador RD. Os dados lidos são enviados para a ULA, que realiza a operação de acordo com o Opcode da instrução. Em sequência, o valor de saída da ULA é selecionado para ser usado para escrita.

Enquanto isso ocorre, o valor do PC é incrementado em 1 para acessar a próxima instrução, passa pelo Mux PC e é enviado para atualizar o valor do PC.

Figura 3 – Caminho de dados - Formato 1 (add, sub, mul, div, and, or, xnor)



Fonte: o Autor.

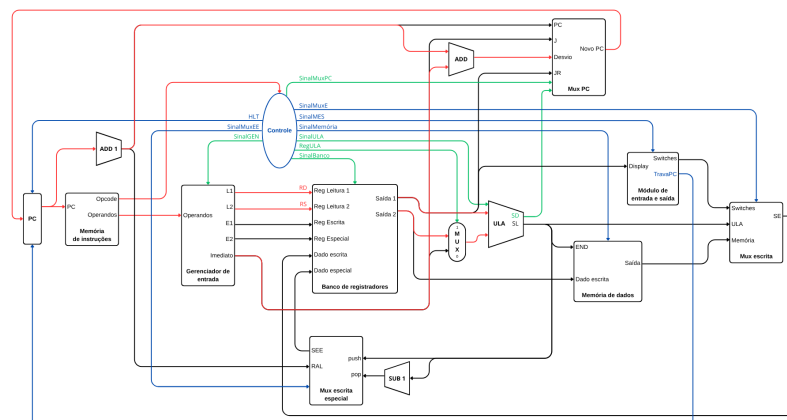
A Figura 4 apresenta o caminho de dados para desvios condicionais tomados. Nela, é possível observar o caminho de dados para as instruções beq, bne, blt, bgt, blet e lget.

Nela é lida a instrução com base no endereço do PC e definidos dois registradores, RD e RS, para leitura e um imediato usado para o desvio.

Nesse caso, o SinalBanco é usado para indicar que não haverá escrita. Sendo assim, são apenas lidos dois registradores do banco de registradores e enviados à ULA para serem comparados.

Em sequência, a ULA envia o sinal de controle SD para indicar que o desvio será tomado e, após isso, o valor do PC é atualizado de acordo com o valor do PC atual somado com o valor do imediato passado.

Figura 4 – Caminho de dados - Formato 3 desvio tomado (beq, bne, blt, bgt)



Fonte: o Autor.

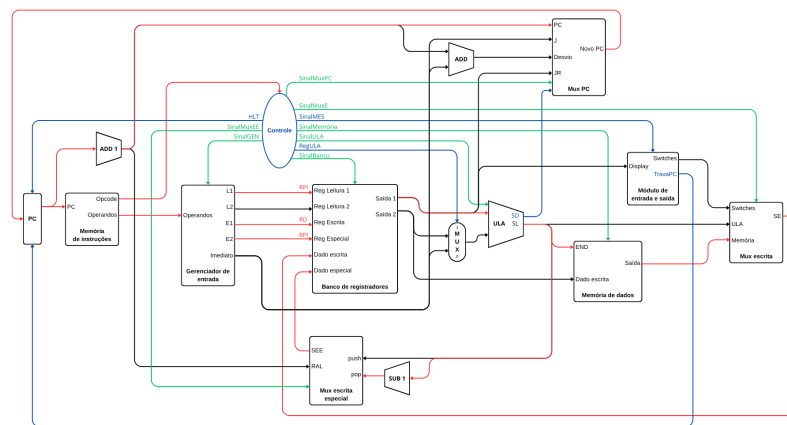
Por fim, a Figura 5 apresenta o caminho de dados para a instrução pop. Nele, é lida

a instrução na memória de instruções, em seguida, o gerenciador de entrada determina o registrador especial RPI tanto para a leitura, para ele fornecer o endereço da pilha, quanto para a escrita, para atualizar o valor dele. Além do registrador RD usado para receber o dado do pop.

O registrador lido é enviado para ULA, onde ele é enviado diretamente para a saída usando o código 01110 presente na Tabela 10. O endereço é enviado para a entrada da memória de dados, onde o SinalMemória determina que será feita uma leitura no endereço. O dado lido passa pelo mux de escrita e é levado para ser armazenado em RD.

Enquanto isso, da saída da ULA o endereço é decrementado em 1, para apontar para a última posição de memória ocupada, passa pelo mux de escrita especial e é usado para atualizar o endereço contido em RPI. Simultaneamente, o PC é incrementado em 1 e atualizado.

Figura 5 – Caminho de dados - Formato 7 (pop)



Fonte: o Autor.

4.5 Descrição dos módulos

4.5.1 ULA

No início da implementação do projeto, foi desenvolvida a unidade de lógica e aritmética (ULA), responsável por fazer as operações de lógica, aritmética e determinar se os desvios vão ou não ocorrer. Todas as entradas e saídas utilizadas, bem como o seu papel, estão presentes na Tabela 9 abaixo.

Tabela 9 – Entradas e saídas do módulo da ULA

ENTRADAS		
Sinal	Nº bits	Descrição
EA	32	Primeiro operando
EB	32	Segundo operando
SinalULA	5	Sinal de controle que define a operação da ULA
SAÍDAS		
Sinal	Nº bits	Descrição
SL	32	Saída padrão com o resultado da operação
SD	1	Indica desvio condicional (1=sim, 0=não)

Fonte: o Autor.

A [Tabela 10](#), abaixo, segue a nomenclatura apresentada na [Tabela 9](#) e apresenta detalhadamente quais são as instruções que utilizam cada um dos valores de SinalULA, bem como qual é a operação realizada em cada um dos casos.

Tabela 10 – Operações realizadas pela ULA conforme o SinalULA

SinalULA	Instrução	Operação
00000	add	$SL = EA + EB$
00001	sub	$SL = EA - EB$
00010	and	$SL = EA \& EB$
00011	or	$SL = EA EB$
00100	not	$SL = \sim EA$
00101	xnor	$SL = EA \sim^{\wedge} EB$
00110	sl	$SL = EA \ll EB[4:0]$
00111	sr	$SL = EA \gg EB[4:0]$
01000	beq	$SD = (EA == EB)$
01001	bne	$SD = (EA != EB)$
01010	blt	$SD = (EA < EB)$
01011	bgt	$SD = (EA > EB)$
01100	mul	$SL = EA * EB$
01101	div	$SL = EA / EB$
01110	pop/mov EA	$SL = EA$
01111	movi	$SL = EB$
10000	push	$SL = EA + 1$

Fonte: o Autor.

Note que existem instruções especiais nas três últimas linhas da [Tabela 10](#), nas quais a operação realizada é consideravelmente diferente do nome das instruções. Isso ocorre pois para simplificar o processo de implementação dessas instruções foi criada uma operação especial para a ULA.

Para a instrução pop, a entrada EA vai receber como entrada o valor do registrador RPI e deve enviar como saída o próprio valor de RPI inalterado para acessar a pilha. No

caso do mov, a entrada EA vai receber o valor de um registrador e a saída deve ser o mesmo valor da entrada para ser salvo em outro registrador.

No caso do movi, a entrada EB vai receber um imediato e esse imediato deverá ser salvo em um registrador, logo a saída da ULA deve ser o próprio EB. Para o push, de modo similar ao pop, a entrada EA recebe o valor de RPI, porém é preciso incrementar em 1 esse valor para acessar corretamente os endereços da pilha, portanto, nesse caso a saída é EA somado com 1.

Além do mais, SD só é utilizado em operações de desvio condicional. Em operações que não são de desvio condicional, a saída SD permanece com o sinal zero. Do mesmo modo, em operações de desvio condicional, o valor da saída SL permanece como zero.

4.5.2 Banco de registradores

O banco de registradores tem o papel de definir quais serão os registradores lidos e/ou escritos. Todas as entradas e saídas utilizadas estão presentes na [Tabela 11](#).

Tabela 11 – Entradas e saídas do módulo do banco de registradores

ENTRADAS		
Sinal	Nº bits	Descrição
clock	1	Controla a escrita, feita somente na borda de subida
reg_leitura1	6	Endereço do primeiro registrador a ser lido
reg_leitura2	6	Endereço do segundo registrador a ser lido
reg_escrita	6	Endereço do registrador de escrita
reg_especial	6	Endereço do registrador especial
dado_escrita	32	Valor a ser escrito no registrador de escrita
dado_especial	32	Valor a ser escrito no registrador especial
SinalBanco[0]	1	Habilita escrita no registrador de escrita (1=sim, 0=não)
SinalBanco[1]	1	Habilita escrita no registrador especial (1=sim, 0=não)
SAÍDAS		
Sinal	Nº bits	Descrição
saida1	32	Valor lido no primeiro registrador
saida2	32	Valor lido no segundo registrador

Fonte: o Autor.

Note que na implementação há um único “SinalBanco” de 2 bits, porém na tabela seus bits foram apresentados separadamente. Isso foi feito para evidenciar que o valor dos bits são independentes entre si, com o bit menos significativo, [0], sendo responsável pela escrita e o mais significativo, [1], responsável pela escrita especial.

Além disso, observe que o registrador de escrita recebe exclusivamente o valor de dado_escrita e o registrador especial recebe exclusivamente o valor de dado_especial. Ademais, ambas as escritas ocorrem somente na borda de subida do clock.

Sobre as saídas, ambas as saídas são feitas de modo assíncrono, ou seja, não dependem do sinal do clock. A saída1 sempre envia o valor do primeiro registrador lido e a saída2 o valor do segundo registrador lido.

O banco foi desenvolvido como uma matriz de 32 posições, correspondentes aos 32 bits de cada registrador, por 64, correspondentes ao número de registradores.

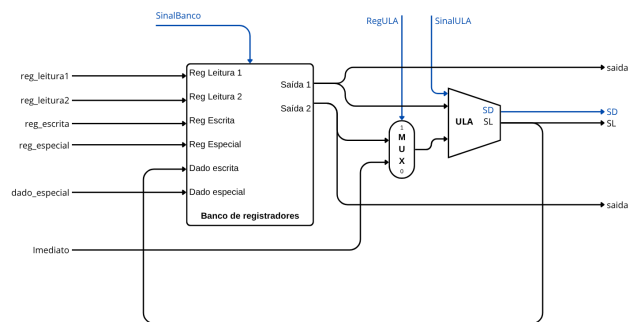
4.5.3 Integração do banco de registrador e ULA

Para essa etapa foi feito um recorte de três módulos do caminho de dados: o banco de registradores, a ULA e o multiplexador. Desse modo, foi preciso usar os dois módulos desenvolvidos anteriormente e criar um novo para o multiplexador.

Primeiramente, sobre o multiplexador, ele é um módulo que recebe dois valores de 32 bits e um bit de controle, que define se a saída, também de 32 bits, será o primeiro valor recebido (1) ou o segundo (0).

A Figura 6 abaixo apresenta o caminho de dados utilizado para a implementação desse módulo. Note que ele é similar ao caminho de dados do processador, presente na Figura 2, porém foram removidos alguns módulos e colocadas algumas saídas a mais para acompanhar o valor lido pelo banco.

Figura 6 – Caminho de dados - Formato 7 (pop)



Fonte: o Autor.

Como sinais de entrada e saída, foram utilizados quase todos os sinais usados no módulo da ULA e no módulo do banco de registradores, presentes na Tabela 9 e Tabela 11.

As entradas não utilizadas presentes na Tabela 9 são a EA e EB, pois os valores desses campos vêm da leitura do banco e do imediato. Além disso, não foi utilizada a entrada dado_escrita da Tabela 11, pois os valores desses campos são providos diretamente da saída da ULA. Todos os sinais de saída foram usados.

Além disso, todos os sinais utilizados neste módulo se comportam da mesma forma que estão descritos na [Tabela 9](#) e [Tabela 11](#).

4.5.4 Memória principal

A memória principal, também chamada de memória de dados, é responsável por armazenar o valor de variáveis durante a execução dos códigos, além de conter a pilha do processador. Os sinais de entrada e saída deste módulo estão presentes na [Tabela 12](#) abaixo.

Tabela 12 – Entradas e saídas do módulo da memória principal

ENTRADAS		
Sinal	Nº bits	Descrição
clock	1	Controla a escrita, feita somente na borda de subida
END	6	Endereço com a posição da memória a ser acessada
dado_escrita	32	Valor a ser escrito no endereço
SinalMemoria	1	Controla a escrita no endereço (1=sim, 0=não)
SAÍDAS		
Sinal	Nº bits	Descrição
saída	32	Valor lido no endereço

Fonte: o Autor.

Note que a escrita ocorre somente na borda de subida do clock e se o valor de SinalMemoria for 1. Por outro lado, a leitura é feita de modo assíncrono, com a saída sendo dada pelo valor no endereço.

Além disso, a memória principal foi criada como uma matriz de 64 linhas de 32 bits. A princípio, foi escolhido o valor de 64 linhas para testar, pois o tempo de compilação não é muito longo. Os 32 bits é devido ao tamanho dos registradores que são de 32 bits cada um.

4.5.5 Memória de instruções

A memória de instruções contém os códigos a serem executados pelo processador. As entradas e saídas desse módulo estão na [Tabela 13](#) abaixo.

Tabela 13 – Entradas e saídas do módulo da memória de instruções

ENTRADAS		
Sinal	Nº bits	Descrição
clock	1	Controla a saída, atualizada na borda de descida
PC	6	Endereço com a posição da memória a ser acessada
SAÍDAS		
Sinal	Nº bits	Descrição
Opcode	6	Opcode da instrução lida na memória
Operandos	26	A instrução sem o Opcode

Fonte: o Autor.

Primeiramente, o valor da saída é atualizado somente na borda de descida do clock. Isso é feito para garantir o funcionamento correto do processador, para que seja possível, nos demais módulos, processar a instrução e realizar as escritas na borda de subida do clock.

Além disso, a saída é dividida em dois campos: Opcode e Operandos. Isso é feito pois o Opcode será enviado diretamente para a unidade de controle, enquanto a saída com os operandos será enviada para o gerenciador de entrada.

Ademais, as instruções são lidas de um arquivo chamado “single_port_rom_init.txt”, sendo assim, elas não são passadas diretamente pelo código em Verilog. Esse arquivo deve conter uma instrução por linha e todas as instruções devem ter 32 bits.

Além disso, foi tomado o valor do Opcode no instante inicial como sendo 000001, de modo a garantir que não seja executada uma instrução inválida ao iniciar o processador. Esse Opcode em questão corresponde à instrução nop, portanto, nada será executado, apenas passará um ciclo de clock.

4.5.6 PC

O módulo do PC controla o endereço da instrução a ser lida na memória de instruções. Para isso, ele atualiza o valor do PC de acordo com o novo valor que ele deve receber. Os sinais de entrada e saída estão presentes na [Tabela 14](#) abaixo.

Tabela 14 – Entradas e saídas do módulo PC

ENTRADAS		
Sinal	Nº bits	Descrição
clock	1	Controla a saída, atualizada na borda de subida
Novo_Endereco	6	Endereço o novo valor do PC
HLT	1	Impede a atualização do PC (1=sim, 0=não)
TravaPC	1	Impede o PC atualizar até um botão ser pressionado (1=sim, 0=não)
SAÍDAS		
Sinal	Nº bits	Descrição
PC	6	Saída com o novo valor do PC

Fonte: o Autor.

A atualização do endereço do PC ocorre sempre na borda de descida do clock e somente se os valores tanto de HLT quanto de TravaPC forem baixos. O sinal HLT provém da unidade de controle e é gerado pela instrução de mesmo nome que interrompe o processador. Enquanto TravaPC é um sinal de controle enviado pelo módulo de entrada e saída enquanto ele espera que um valor seja inserido nos switches.

Após assumir o valor 1, o sinal HLT não pode voltar a ser zero, a não ser que o processador seja resetado. Por outro lado, o TravaPC é desativado quando um botão for pressionado, indicando que já foi colocado o valor desejado nos switches, então, eles são lidos e o código continua.

4.5.7 Unidade de controle

O papel da unidade de controle é, a partir dos Opcodes das instruções, determinar qual deve ser o sinal de controle para cada um dos sinais de controle. Isso é feito utilizando um case para cada uma das instruções e definindo qual deve ser o valor de cada sinal de controle para essa instrução funcionar corretamente.

A [Tabela 15](#) apresenta todos os sinais de entrada e saída que a unidade de controle utiliza, bem como o número de bits de cada um e a sua descrição. Nela o “X” indica que a respectiva escrita ocorre independente do sinal do outro bit.

Tabela 15 – Entradas e saídas da unidade de controle

ENTRADAS		
Sinal	Nº bits	Descrição
Opcode	6	Valor do Opcode da instrução
SAÍDAS		
Sinal	Nº bits	Descrição
HLT	1	Impede o PC de atualizar
RegULA	1	Escolhe entre imediato [0] e registrador [1] para ULA
SinalMemoria	1	Indica escrita na memória de dados
SinalMES	2	Indica se é uma instrução IN [01] ou OUT [10]
SinalBanco	2	Controla escrita [X1] e escrita especial [1X]
SinalMuxE	2	Define qual será o dado de escrita
SinalMuxEE	2	Define qual será o dado de escrita especial
SinalMuxPC	2	Define o próximo valor do PC
SinalGEN	3	Define como serão obtido os operando da instrução
SinalULA	5	Operação a ser realizada pela ULA

Fonte: o Autor.

4.5.8 Módulo de entrada e saída

O módulo de entrada e saída é responsável por receber os valores lidos pelos switches e enviá-los para o mux de escrita. Durante a leitura, o módulo também é responsável pelo sinal de controle TravaPC, responsável por parar a atualização do PC até que um botão de controle seja pressionado, indicando que foi escrito o valor desejado nos switches.

Além do mais, também é responsável por receber um registrador que deve ter o seu conteúdo lido. Para isso, implementa um decodificador que, a partir de um valor em binário de 32 bits, identifica quais são cada um dos dígitos desse valor ao ser convertido para decimal. Ademais, a partir desses dígitos em decimal, escreve em cada um dos oito displays de sete segmentos.

A [Tabela 16](#) apresenta os sinais de entrada e saída do módulo de entrada e saída, bem como o número de bits de cada um e a descrição. Nela o “Display_X” foi escrito dessa forma pois, ao todo, há 8 displays e, para cada um deles, há um sinal de saída. Desse modo, foi usado “X” para indicar cada uma dessas 8 possibilidades.

Tabela 16 – Entradas e saídas do módulo de entrada e saída

ENTRADAS		
Sinal	Nº bits	Descrição
clock	1	Sinal de clock
SinalMES	2	Controla a leitura (bit 0) e a escrita (bit 1)
Display_IN	32	Valor a ser escrito no display
Switches_IN	18	Valor lido dos 18 switches
Destrava	1	Botão que destrava o PC
SAÍDAS		
Sinal	Nº bits	Descrição
Switches_OUT	32	Valor dos switches estendido
TravaPC	1	Sinal que impede o PC de atualizar
Display_X	7	Valor a ser escrito no display X

Fonte: o Autor.

Nesse módulo o sinal TravaPC se torna 1 sempre que é executada uma instrução IN. Além disso, permanece como 1 até que seja pressionado o botão destrava.

Para o funcionamento correto do botão, foi necessário implementar um debouncer para garantir que o TravaPC só volte a ser zero uma única vez a cada vez que o botão for pressionado. Isso foi necessário pois, caso contrário, um único pulso no botão poderia tomar a mesma configuração de switches como sinal de entrada para várias instruções IN em sequência.

Além disso, dentro do módulo também foi criado um buffer para que o valor escrito no display só seja atualizado nas instruções OUT e, nas demais instruções, permaneça com seu valor inalterado.

Ademais, no mesmo documento em que foi desenvolvido o módulo de entrada e saída, também foi criado um módulo para converter dígitos de binário para o display de sete segmentos por meio de um case.

4.5.9 Integração final

A integração final tem o papel de unir o conteúdo de todos os módulos desenvolvidos de modo que sejam capazes de se comunicarem entre si e, dessa forma, simular o comportamento de um processador usando a linguagem Verilog.

Para isso, foi necessário criar um arquivo em Verilog e, a partir dele, chamar a todos os outros módulos desenvolvidos, passando as respectivas variáveis de entrada e saída para cada um deles.

A [Tabela 17](#) apresenta os sinais de entrada e saída da integração final de todos os módulos, bem como o número de bits de cada um e a descrição. Nela, de modo similar aos

sinais de saída do módulo de entrada e saída, o sinal “Display_X” foi escrito dessa forma com o “X” representando os 8 displays.

Tabela 17 – Entradas e saídas da integração final

ENTRADAS		
Sinal	Nº bits	Descrição
clock	1	Sinal de clock do FPGA 50MHz
Destrava	1	Botão para destravar o PC
Switches_IN	18	Valor de entrada dos 18 switches
SAÍDAS		
Sinal	Nº bits	Descrição
Display_X	7	Valor a ser escrito no display X

Fonte: o Autor.

Nesse módulo também foi desenvolvido um divisor de frequência de 50MHz para 1MHz, com somente esse sinal com a frequência reduzida sendo passado para os demais módulos.

4.6 Implementações

4.6.1 ULA

No Código 4.1 abaixo é apresentado o código desenvolvido para a ULA.

```

1 module ULA (
2     input wire [31:0] EA,
3     input wire [31:0] EB,
4     input wire [4:0] SinalULA,
5     output reg [31:0] SL,
6     output reg SD
7 );
8
9 // Atribuicao inicial para evitar latches (logica combinacional pura)
10 always @(*) begin
11     SL = 32'd0;
12     SD = 1'b0;
13
14     case(SinalULA)
15         5'b00000: SL = EA + EB;
16         5'b00001: SL = EA - EB;
17         5'b00010: SL = EA & EB;
18         5'b00011: SL = EA | EB;
19         5'b00100: SL = ~EA;
20         5'b00101: SL = EA ^ EB;
21         5'b00110: SL = EA << EB[4:0];
22         5'b00111: SL = EA >> EB[4:0];
23         5'b01000: SD = (EA == EB);
24         5'b01001: SD = (EA != EB);
25         5'b01010: SD = (EA < EB);
26         5'b01011: SD = (EA > EB);

```

```
27      5'b01100: SL = EA * EB;
28      5'b01101: SL = EA / EB;
29      5'b01110: SL = EA;
30      5'b01111: SL = EB;
31      5'b10000: SL = EA + 1;
32      5'b10001: SD = (EA <= EB);
33      5'b10010: SD = (EA >= EB);
34      default: begin
35          SL = 32'd0;
36          SD = 1'b0;
37      end
38  endcase
39 end
40
41 endmodule
```

Código 4.1 – Módulo da ULA em Verilog

4.6.2 Banco de registradores

No [Código 4.2](#) abaixo é apresentado o código desenvolvido para o banco de registradores.

```

1 module Banco_de_Registradores
2   #(parameter ADDR_WIDTH = 6)           // 2^4 = 16 registradores (mude para 5
      para 32, ou 6 para 64)
3   (clock, reg_leitura1, reg_leitura2, reg_escrita, reg_especial, dado_escrita,
      dado_especial, SinalBanco, saida1, saida2);
4
5   input clock;                          // Clock do
      processador
6   input [5:0] reg_leitura1;              // Endereco do registrador 1
7   input [5:0] reg_leitura2;              // Endereco do registrador 2
8   input [5:0] reg_escrita;               // Endereco de escrita
9   input [5:0] reg_especial;              // Endereco de escrita especial
10  input [31:0] dado_escrita;              // Dado para escrever
11  input [31:0] dado_especial;             // Dado para escrever especial
12
13  input [1:0] SinalBanco;                 // [00] = sem escrita, [01] =
      escrita padrao, [10] = escrita especial, [11] = ambos
14
15  output [31:0] saida1;                   // Dado de saida 1
16  output [31:0] saida2;                   // Dado de saida 2
17
18  reg [31:0] banco [2**ADDR_WIDTH-1:0];   // Criacao do banco de
      registradores
19
20  always @(posedge clock)
21  begin
22      // Garante que o registrador 0 (6'd0) nunca seja sobrescrito
23      if(SinalBanco[0] == 1'b1 && reg_escrita[ADDR_WIDTH-1:0] != 0)
24          banco[reg_escrita[ADDR_WIDTH-1:0]] <= dado_escrita;
25
26      if(SinalBanco[1] == 1'b1 && reg_especial[ADDR_WIDTH-1:0] != 0)
27          banco[reg_especial[ADDR_WIDTH-1:0]] <= dado_especial;
28  end
29
30  // Se a leitura for no Endereco 0, a saida e 0. Caso contrario, le o banco normalmente.
31  assign saida1 = (reg_leitura1[ADDR_WIDTH-1:0] == 0) ? 32'd0 : banco[reg_leitura1[
      ADDR_WIDTH-1:0]];
32  assign saida2 = (reg_leitura2[ADDR_WIDTH-1:0] == 0) ? 32'd0 : banco[reg_leitura2[
      ADDR_WIDTH-1:0]];
33
34  endmodule

```

Código 4.2 – Módulo do banco de registradores em Verilog

4.6.3 Integração ULA com banco de registradores

No [Código 4.3](#) abaixo é apresentado o código desenvolvido para a integração da ULA com o banco de registradores.

```

1 module Integracao_ULA_Banco (
2 // Entradas e saidas para o banco
3 input clock, // Clock do processador
4 input [5:0] reg_leitura1, // Endereco do registrador 1
5 input [5:0] reg_leitura2, // Endereco do registrador 2
6 input [5:0] reg_escrita, // Endereco de escrita
7 input [5:0] reg_especial, // Endereco de escrita especial
8 input [31:0] dado_especial, // Dado para escrever especial
9 input [1:0] SinalBanco, // Controla a escrita: bit[1] =
    especial, bit[0] = escrita
10 output [31:0] saida1, // Valor do primeiro registrador
    lido
11 output [31:0] saida2, // Valor do segundo registrador
    lido
12
13 // Entradas e saidas para ULA
14 input [4:0] SinalULA, // Codigo de operacao da ULA
15 output [31:0] SL, // Saida padrao
16 output SD, // Saida para desvio
17
18 // Entrada de imediato
19 input [31:0] Imediato, // Imediato
20 input RegULA // Sinal que define se e imediato
    (0) ou reg (1)
21 );
22
23 wire [31:0] EA;
24 wire [31:0] EB;
25
26 // Envia os dados para o banco
27 Banco_de_Registradores #(ADDR_WIDTH(6)) banco_inst (
28     .clock(clock),
29     .reg_leitura1(reg_leitura1),
30     .reg_leitura2(reg_leitura2),
31     .reg_escrita(reg_escrita),
32     .reg_especial(reg_especial),
33     .dado_escrita(SL), // Saida padrao da ULA
34     .dado_especial(dado_especial),
35     .SinalBanco(SinalBanco),
36     .saida1(saida1),
37     .saida2(saida2)
38 );
39
40 // Atribui os valores de entrada para ULA
41 // Primeira entrada da ULA vem da saida1 do banco
42 assign EA = saida1;
43
44 // Segunda entrada vem ou da saida2 ou do imediato
45 Mux2(
46     .E1(saida2),
47     .E2(Imediato),
48     .S(EB),
49     .Sinal(RegULA)
50 );

```

```
51
52 // Envia os dados para ULA
53 ULA (
54     .EA(EA),
55     .EB(EB),
56     .SinalULA(SinalULA),
57     .SL(SL),
58     .SD(SD)
59 );
60
61 endmodule
```

Código 4.3 – Módulo da integração da ULA com o banco de registradores em Verilog

4.6.4 Memória principal

No Código 4.4 abaixo é apresentado o código desenvolvido para a memória principal.

```
1 // Quartus Prime Verilog Template
2
3 module Memoria_Principal
4 #(parameter DATA_WIDTH=32, parameter ADDR_WIDTH=8) // 32 bits e 2^8 = 256 linhas
5 (
6     input clock,
7         // Sinal de clock
8     input [(ADDR_WIDTH-1):0] END, // Endereco de
        acesso
9     input [(DATA_WIDTH-1):0] dado_escrita, // Dado para escrever
10    input SinalMemoria,
        // Sinal de escrita
11    output reg [(DATA_WIDTH-1):0] saida // Dado de saida
12);
13
14    // Criacao da memoria
15    reg [DATA_WIDTH-1:0] ram[2**ADDR_WIDTH-1:0];
16
17    always @ (negedge clock)
18    begin
19        // Escrita na memoria
20        if (SinalMemoria) ram[END] <= dado_escrita;
21
22        // Leitura
23        saida <= ram[END];
24    end
25 endmodule
```

Código 4.4 – Módulo da memória principal em Verilog

4.6.5 Memória instruções

No [Código 4.5](#) abaixo é apresentado o código desenvolvido para a memória de instruções.

```
1 // Quartus Prime Verilog Template
2 // Single Port ROM
3
4 module Memoria_Instrucoes
5 #(parameter DATA_WIDTH=32, parameter ADDR_WIDTH=8)
6 (
7     input clock,
8     input [(ADDR_WIDTH-1):0] PC,
9     output reg [5:0] Opcode,
10    output reg [25:0] Operandos
11 );
12
13    // Declare the ROM variable
14    reg [DATA_WIDTH-1:0] rom[2**ADDR_WIDTH-1:0];
15
16    // Initialize the ROM with $readmemb. Put the memory contents
17    // in the file single_port_rom_init.txt. Without this file,
18    // this design will not compile.
19
20    // See Verilog LRM 1364-2001 Section 17.2.8 for details on the
21    // format of this file, or see the "Using $readmemb and $readmemh"
22    // template later in this section.
23
24    initial
25    begin
26        $readmemb("single_port_rom_init.txt", rom);
27        Opcode = 6'd1; // Forca Opcode inicial como 000001 (nop)
28        Operandos = 26'd0; // Apenas para atribuir um valor
29    end
30
31
32    always @ (posedge clock)
33    begin
34        Opcode <= rom[PC][31:26];
35        Operandos <= rom[PC][25:0];
36    end
37
38 endmodule
```

Código 4.5 – Módulo da memória de instruções em Verilog

4.6.6 PC

No [Código 4.6](#) abaixo é apresentado o código desenvolvido para o módulo do PC.

```
1 module PC(clock, Novo_Endereco, HLT, TravaPC, PC);
2
3 input clock;                                // Sinal de clock
4 input [7:0] Novo_Endereco;                  // Novo valor do PC
5 input HLT;                                  // Sinal de encerrar
6 input TravaPC;                              // Sinal para segurar para ler
   switches
7 output reg [7:0] PC;                        // Saida do PC
8
9 reg aux = 1'b0;
10
11 always @(negedge clock)
12 begin
13     if(aux == 1'b0)
14     begin
15         PC = 7'd0;
16         aux = 1'b1;
17     end
18
19     if(!HLT & !TravaPC)
20     begin
21         PC = Novo_Endereco;
22     end
23 end
24
25 endmodule
```

Código 4.6 – Módulo do PC em Verilog

4.6.7 Unidade de controle

No [Código 4.7](#) abaixo é apresentado o código desenvolvido para o módulo da unidade de controle.

```

1  module Unidade_de_Controle(
2  input [5:0] Opcode,
3  output reg HLT,
4  output reg RegULA,
5  output reg SinalMemoria,
6  output reg [1:0] SinalMES,
7  output reg [1:0] SinalBanco,
8  output reg [1:0] SinalMuxE,
9  output reg [1:0] SinalMuxEE,
10 output reg [1:0] SinalMuxPC,
11 output reg [2:0] SinalGEN,
12 output reg [4:0] SinalULA
13 );
14
15 always @(*)
16 begin
17     SinalMES = 2'b00;
18
19     case(Opcode)
20         // Formato 7 - hlt
21         6'b000000:
22             begin
23                 HLT = 1'b1;
24                 RegULA = 1'b0; // Registrador para ULA
25                 SinalMemoria = 1'b0; // Controla escrita
26                 SinalMES = 2'b00; // [10] = OUT, [01] = IN, [00] = default
27                 SinalBanco = 2'b00; // [00] = sem escrita, [01] =
28                 // escrita padrao, [10] = escrita especial, [11] = ambos
29                 SinalMuxE = 2'b00; // [00] = Switches, [01] = ULA,
30                 // [10] = Memoria
31                 SinalMuxEE = 2'b00; // [00] = RAL, [01] = Push, [10] = Pop
32                 SinalMuxPC = 2'b00; // [00] = PC, [01] = J, [10] =
33                 // Desvio, [11] = JR
34                 SinalGEN = 3'b110; // Num Formato - 1
35                 SinalULA = 5'b11111;
36             end
37         // Formato 7 - nop
38         6'b000001:
39             begin
40                 HLT = 1'b0;
41                 RegULA = 1'b0; // Registrador para ULA
42                 SinalMemoria = 1'b0; // Controla escrita
43                 SinalMES = 2'b00; // [10] = OUT, [01] = IN, [00] = default
44                 SinalBanco = 2'b00; // [00] = sem escrita, [01] =
45                 // escrita padrao, [10] = escrita especial, [11] = ambos
46                 SinalMuxE = 2'b00; // [00] = Switches, [01] = ULA,
47                 // [10] = Memoria
48                 SinalMuxEE = 2'b00; // [00] = RAL, [01] = Push, [10] = Pop
49                 SinalMuxPC = 2'b00; // [00] = PC, [01] = J, [10] =
50                 // Desvio, [11] = JR
51                 SinalGEN = 3'b110; // Num Formato - 1
52                 SinalULA = 5'b11111;
53             end
54         // Formato 7 - jmp

```

```

49      6'b000010:
50      begin
51          HLT = 1'b0;
52          RegULA = 1'b0;                // Registrador para ULA
53          SinalMemoria = 1'b0;          // Controla escrita
54          SinalMES = 2'b00;             // [10] = OUT, [01] = IN, [00] = default
55          SinalBanco = 2'b00;           // [00] = sem escrita, [01] =
56          escrita padrao, [10] = escrita especial, [11] = ambos
57          SinalMuxE = 2'b00;            // [00] = Switches, [01] = ULA,
58          [10] = Memoria
59          SinalMuxEE = 2'b00;           // [00] = RAL, [01] = Push, [10] = Pop
60          SinalMuxPC = 2'b01;           // [00] = PC, [01] = J, [10] =
61          Desvio, [11] = JR
62          SinalGEN = 3'b110;            // Num Formato - 1
63          SinalULA = 5'b11111;
64      end
65      // Formato 7 - jal
66      6'b000011:
67      begin
68          HLT = 1'b0;
69          RegULA = 1'b0;                // Registrador para ULA
70          SinalMemoria = 1'b0;          // Controla escrita
71          SinalMES = 2'b00;             // [10] = OUT, [01] = IN, [00] = default
72          SinalBanco = 2'b10;           // [00] = sem escrita, [01] =
73          escrita padrao, [10] = escrita especial, [11] = ambos
74          SinalMuxE = 2'b00;            // [00] = Switches, [01] = ULA,
75          [10] = Memoria
76          SinalMuxEE = 2'b00;           // [00] = RAL, [01] = Push, [10] = Pop
77          SinalMuxPC = 2'b01;           // [00] = PC, [01] = J, [10] =
78          Desvio, [11] = JR
79          SinalGEN = 3'b110;            // Num Formato - 1
80          SinalULA = 5'b11111;
81      end
82      // Formato 1 - add
83      6'b000100:
84      begin
85          HLT = 1'b0;
86          RegULA = 1'b1;                // Registrador para ULA
87          SinalMemoria = 1'b0;          // Controla escrita
88          SinalMES = 2'b00;             // [10] = OUT, [01] = IN, [00] = default
89          SinalBanco = 2'b01;           // [00] = sem escrita, [01] =
90          escrita padrao, [10] = escrita especial, [11] = ambos
91          SinalMuxE = 2'b01;            // [00] = Switches, [01] = ULA,
92          [10] = Memoria
93          SinalMuxEE = 2'b00;           // [00] = RAL, [01] = Push, [10] = Pop
94          SinalMuxPC = 2'b00;           // [00] = PC, [01] = J, [10] =
95          Desvio, [11] = JR
96          SinalGEN = 3'b000;            // Num Formato - 1
97          SinalULA = 5'b00000;
98      end
99      // Formato 1 - sub
100     6'b000101:
101     begin
102         HLT = 1'b0;
103         RegULA = 1'b1;                // Registrador para ULA
104         SinalMemoria = 1'b0;          // Controla escrita
105         SinalMES = 2'b00;             // [10] = OUT, [01] = IN, [00] = default
106         SinalBanco = 2'b01;           // [00] = sem escrita, [01] =
107         escrita padrao, [10] = escrita especial, [11] = ambos

```

```

98         SinalMuxE = 2'b01;           // [00] = Switches, [01] = ULA,
           [10] = Memoria
99         SinalMuxEE = 2'b00;         // [00] = RAL, [01] = Push, [10] = Pop
100        SinalMuxPC = 2'b00;           // [00] = PC, [01] = J, [10] =
           Desvio, [11] = JR
101        SinalGEN = 3'b000;           // Num Formato - 1
102        SinalULA = 5'b000001;
103    end
104    // Formato 1 - mul
105    6'b000110:
106    begin
107        HLT = 1'b0;
108        RegULA = 1'b1;                // Registrador para ULA
109        SinalMemoria = 1'b0;         // Controla escrita
110        SinalMES = 2'b00;            // [10] = OUT, [01] = IN, [00] = default
111        SinalBanco = 2'b01;          // [00] = sem escrita, [01] =
           escrita padrao, [10] = escrita especial, [11] = ambos
112        SinalMuxE = 2'b01;           // [00] = Switches, [01] = ULA,
           [10] = Memoria
113        SinalMuxEE = 2'b00;          // [00] = RAL, [01] = Push, [10] = Pop
114        SinalMuxPC = 2'b00;          // [00] = PC, [01] = J, [10] =
           Desvio, [11] = JR
115        SinalGEN = 3'b000;           // Num Formato - 1
116        SinalULA = 5'b01100;
117    end
118    // Formato 1 - div
119    6'b000111:
120    begin
121        HLT = 1'b0;
122        RegULA = 1'b1;                // Registrador para ULA
123        SinalMemoria = 1'b0;         // Controla escrita
124        SinalMES = 2'b00;            // [10] = OUT, [01] = IN, [00] = default
125        SinalBanco = 2'b01;          // [00] = sem escrita, [01] =
           escrita padrao, [10] = escrita especial, [11] = ambos
126        SinalMuxE = 2'b01;           // [00] = Switches, [01] = ULA,
           [10] = Memoria
127        SinalMuxEE = 2'b00;          // [00] = RAL, [01] = Push, [10] = Pop
128        SinalMuxPC = 2'b00;          // [00] = PC, [01] = J, [10] =
           Desvio, [11] = JR
129        SinalGEN = 3'b000;           // Num Formato - 1
130        SinalULA = 5'b01101;
131    end
132    // Formato 1 - and
133    6'b001000:
134    begin
135        HLT = 1'b0;
136        RegULA = 1'b1;                // Registrador para ULA
137        SinalMemoria = 1'b0;         // Controla escrita
138        SinalMES = 2'b00;            // [10] = OUT, [01] = IN, [00] = default
139        SinalBanco = 2'b01;          // [00] = sem escrita, [01] =
           escrita padrao, [10] = escrita especial, [11] = ambos
140        SinalMuxE = 2'b01;           // [00] = Switches, [01] = ULA,
           [10] = Memoria
141        SinalMuxEE = 2'b00;          // [00] = RAL, [01] = Push, [10] = Pop
142        SinalMuxPC = 2'b00;          // [00] = PC, [01] = J, [10] =
           Desvio, [11] = JR
143        SinalGEN = 3'b000;           // Num Formato - 1
144        SinalULA = 5'b00010;
145    end
146    // Formato 1 - or

```

```

147      6'b001001:
148      begin
149          HLT = 1'b0;
150          RegULA = 1'b1;                // Registrador para ULA
151          SinalMemoria = 1'b0;          // Controla escrita
152          SinalMES = 2'b00;             // [10] = OUT, [01] = IN, [00] = default
153          SinalBanco = 2'b01;           // [00] = sem escrita, [01] =
154          escrita padrao, [10] = escrita especial, [11] = ambos
155          SinalMuxE = 2'b01;            // [00] = Switches, [01] = ULA,
156          [10] = Memoria
157          SinalMuxEE = 2'b00;           // [00] = RAL, [01] = Push, [10] = Pop
158          SinalMuxPC = 2'b00;           // [00] = PC, [01] = J, [10] =
159          Desvio, [11] = JR
160          SinalGEN = 3'b000;            // Num Formato - 1
161          SinalULA = 5'b00011;
162      end
163      // Formato 1 - xnor
164      6'b001010:
165      begin
166          HLT = 1'b0;
167          RegULA = 1'b1;                // Registrador para ULA
168          SinalMemoria = 1'b0;          // Controla escrita
169          SinalMES = 2'b00;             // [10] = OUT, [01] = IN, [00] = default
170          SinalBanco = 2'b01;           // [00] = sem escrita, [01] =
171          escrita padrao, [10] = escrita especial, [11] = ambos
172          SinalMuxE = 2'b01;            // [00] = Switches, [01] = ULA,
173          [10] = Memoria
174          SinalMuxEE = 2'b00;           // [00] = RAL, [01] = Push, [10] = Pop
175          SinalMuxPC = 2'b00;           // [00] = PC, [01] = J, [10] =
176          Desvio, [11] = JR
177          SinalGEN = 3'b000;            // Num Formato - 1
178          SinalULA = 5'b00101;
179      end
180      // Formato 2 - addi
181      6'b001011:
182      begin
183          HLT = 1'b0;
184          RegULA = 1'b0;                // Registrador para ULA
185          SinalMemoria = 1'b0;          // Controla escrita
186          SinalMES = 2'b00;             // [10] = OUT, [01] = IN, [00] = default
187          SinalBanco = 2'b01;           // [00] = sem escrita, [01] =
188          escrita padrao, [10] = escrita especial, [11] = ambos
189          SinalMuxE = 2'b01;            // [00] = Switches, [01] = ULA,
190          [10] = Memoria
191          SinalMuxEE = 2'b00;           // [00] = RAL, [01] = Push, [10] = Pop
192          SinalMuxPC = 2'b00;           // [00] = PC, [01] = J, [10] =
193          Desvio, [11] = JR
194          SinalGEN = 3'b001;            // Num Formato - 1
195          SinalULA = 5'b00000;
196      end
197      // Formato 2 - subi
198      6'b001100:
199      begin
200          HLT = 1'b0;
201          RegULA = 1'b0;                // Registrador para ULA
202          SinalMemoria = 1'b0;          // Controla escrita
203          SinalMES = 2'b00;             // [10] = OUT, [01] = IN, [00] = default
204          SinalBanco = 2'b01;           // [00] = sem escrita, [01] =
205          escrita padrao, [10] = escrita especial, [11] = ambos

```

```

196         SinalMuxE = 2'b01;           // [00] = Switches, [01] = ULA,
           [10] = Memoria
197         SinalMuxEE = 2'b00;          // [00] = RAL, [01] = Push, [10] = Pop
198         SinalMuxPC = 2'b00;          // [00] = PC, [01] = J, [10] =
           Desvio, [11] = JR
199         SinalGEN = 3'b001;           // Num Formato - 1
200         SinalULA = 5'b00001;
201     end
202     // Formato 2 - muli
203     6'b001101:
204     begin
205         HLT = 1'b0;
206         RegULA = 1'b0;                // Registrador para ULA
207         SinalMemoria = 1'b0;          // Controla escrita
208         SinalMES = 2'b00;             // [10] = OUT, [01] = IN, [00] = default
209         SinalBanco = 2'b01;           // [00] = sem escrita, [01] =
           escrita padrao, [10] = escrita especial, [11] = ambos
210         SinalMuxE = 2'b01;           // [00] = Switches, [01] = ULA,
           [10] = Memoria
211         SinalMuxEE = 2'b00;          // [00] = RAL, [01] = Push, [10] = Pop
212         SinalMuxPC = 2'b00;          // [00] = PC, [01] = J, [10] =
           Desvio, [11] = JR
213         SinalGEN = 3'b001;           // Num Formato - 1
214         SinalULA = 5'b01100;
215     end
216     // Formato 2 - divi
217     6'b001110:
218     begin
219         HLT = 1'b0;
220         RegULA = 1'b0;                // Registrador para ULA
221         SinalMemoria = 1'b0;          // Controla escrita
222         SinalMES = 2'b00;             // [10] = OUT, [01] = IN, [00] = default
223         SinalBanco = 2'b01;           // [00] = sem escrita, [01] =
           escrita padrao, [10] = escrita especial, [11] = ambos
224         SinalMuxE = 2'b01;           // [00] = Switches, [01] = ULA,
           [10] = Memoria
225         SinalMuxEE = 2'b00;          // [00] = RAL, [01] = Push, [10] = Pop
226         SinalMuxPC = 2'b00;          // [00] = PC, [01] = J, [10] =
           Desvio, [11] = JR
227         SinalGEN = 3'b001;           // Num Formato - 1
228         SinalULA = 5'b01101;
229     end
230     // Formato 2 - andi
231     6'b001111:
232     begin
233         HLT = 1'b0;
234         RegULA = 1'b0;                // Registrador para ULA
235         SinalMemoria = 1'b0;          // Controla escrita
236         SinalMES = 2'b00;             // [10] = OUT, [01] = IN, [00] = default
237         SinalBanco = 2'b01;           // [00] = sem escrita, [01] =
           escrita padrao, [10] = escrita especial, [11] = ambos
238         SinalMuxE = 2'b01;           // [00] = Switches, [01] = ULA,
           [10] = Memoria
239         SinalMuxEE = 2'b00;          // [00] = RAL, [01] = Push, [10] = Pop
240         SinalMuxPC = 2'b00;          // [00] = PC, [01] = J, [10] =
           Desvio, [11] = JR
241         SinalGEN = 3'b001;           // Num Formato - 1
242         SinalULA = 5'b00010;
243     end
244     // Formato 2 - ori

```

```

245      6'b010000:
246      begin
247          HLT = 1'b0;
248          RegULA = 1'b0;                // Registrador para ULA
249          SinalMemoria = 1'b0;          // Controla escrita
250          SinalMES = 2'b00;             // [10] = OUT, [01] = IN, [00] = default
251          SinalBanco = 2'b01;           // [00] = sem escrita, [01] =
252          escrita padrao, [10] = escrita especial, [11] = ambos
253          SinalMuxE = 2'b01;            // [00] = Switches, [01] = ULA,
254          [10] = Memoria
255          SinalMuxEE = 2'b00;           // [00] = RAL, [01] = Push, [10] = Pop
256          SinalMuxPC = 2'b00;           // [00] = PC, [01] = J, [10] =
257          Desvio, [11] = JR
258          SinalGEN = 3'b001;            // Num Formato - 1
259          SinalULA = 5'b00011;
260      end
261      // Formato 2 - not
262      6'b010001:
263      begin
264          HLT = 1'b0;
265          RegULA = 1'b0;                // Registrador para ULA
266          SinalMemoria = 1'b0;          // Controla escrita
267          SinalMES = 2'b00;             // [10] = OUT, [01] = IN, [00] = default
268          SinalBanco = 2'b01;           // [00] = sem escrita, [01] =
269          escrita padrao, [10] = escrita especial, [11] = ambos
270          SinalMuxE = 2'b01;            // [00] = Switches, [01] = ULA,
271          [10] = Memoria
272          SinalMuxEE = 2'b00;           // [00] = RAL, [01] = Push, [10] = Pop
273          SinalMuxPC = 2'b00;           // [00] = PC, [01] = J, [10] =
274          Desvio, [11] = JR
275          SinalGEN = 3'b001;            // Num Formato - 1
276          SinalULA = 5'b00100;
277      end
278      // Formato 2 - sl
279      6'b010010:
280      begin
281          HLT = 1'b0;
282          RegULA = 1'b0;                // Registrador para ULA
283          SinalMemoria = 1'b0;          // Controla escrita
284          SinalMES = 2'b00;             // [10] = OUT, [01] = IN, [00] = default
285          SinalBanco = 2'b01;           // [00] = sem escrita, [01] =
286          escrita padrao, [10] = escrita especial, [11] = ambos
287          SinalMuxE = 2'b01;            // [00] = Switches, [01] = ULA,
288          [10] = Memoria
289          SinalMuxEE = 2'b00;           // [00] = RAL, [01] = Push, [10] = Pop
290          SinalMuxPC = 2'b00;           // [00] = PC, [01] = J, [10] =
291          Desvio, [11] = JR
292          SinalGEN = 3'b001;            // Num Formato - 1
293          SinalULA = 5'b00110;
294      end
295      // Formato 2 - sr
296      6'b010011:
297      begin
298          HLT = 1'b0;
299          RegULA = 1'b0;                // Registrador para ULA
300          SinalMemoria = 1'b0;          // Controla escrita
301          SinalMES = 2'b00;             // [10] = OUT, [01] = IN, [00] = default
302          SinalBanco = 2'b01;           // [00] = sem escrita, [01] =
303          escrita padrao, [10] = escrita especial, [11] = ambos

```

```

294         SinalMuxE = 2'b01;           // [00] = Switches, [01] = ULA,
        [10] = Memoria
295         SinalMuxEE = 2'b00;          // [00] = RAL, [01] = Push, [10] = Pop
296         SinalMuxPC = 2'b00;          // [00] = PC, [01] = J, [10] =
        Desvio, [11] = JR
297         SinalGEN = 3'b001;           // Num Formato - 1
298         SinalULA = 5'b00111;
299     end
300     // Formato 2 - mov
301     6'b010100:
302     begin
303         HLT = 1'b0;
304         RegULA = 1'b0;                // Registrador para ULA
305         SinalMemoria = 1'b0;          // Controla escrita
306         SinalMES = 2'b00;             // [10] = OUT, [01] = IN, [00] = default
307         SinalBanco = 2'b01;           // [00] = sem escrita, [01] =
        escrita padrao, [10] = escrita especial, [11] = ambos
308         SinalMuxE = 2'b01;           // [00] = Switches, [01] = ULA,
        [10] = Memoria
309         SinalMuxEE = 2'b00;          // [00] = RAL, [01] = Push, [10] = Pop
310         SinalMuxPC = 2'b00;          // [00] = PC, [01] = J, [10] =
        Desvio, [11] = JR
311         SinalGEN = 3'b001;           // Num Formato - 1
312         SinalULA = 5'b01110;
313     end
314     // Formato 2 - loadb
315     6'b010101:
316     begin
317         HLT = 1'b0;
318         RegULA = 1'b0;                // Registrador para ULA
319         SinalMemoria = 1'b0;          // Controla escrita
320         SinalMES = 2'b00;             // [10] = OUT, [01] = IN, [00] = default
321         SinalBanco = 2'b01;           // [00] = sem escrita, [01] =
        escrita padrao, [10] = escrita especial, [11] = ambos
322         SinalMuxE = 2'b10;           // [00] = Switches, [01] = ULA,
        [10] = Memoria
323         SinalMuxEE = 2'b00;          // [00] = RAL, [01] = Push, [10] = Pop
324         SinalMuxPC = 2'b00;          // [00] = PC, [01] = J, [10] =
        Desvio, [11] = JR
325         SinalGEN = 3'b001;           // Num Formato - 1
326         SinalULA = 5'b00000;
327     end
328     // Formato 2 - loadr
329     6'b010110:
330     begin
331         HLT = 1'b0;
332         RegULA = 1'b0;                // Registrador para ULA
333         SinalMemoria = 1'b0;          // Controla escrita
334         SinalMES = 2'b00;             // [10] = OUT, [01] = IN, [00] = default
335         SinalBanco = 2'b01;           // [00] = sem escrita, [01] =
        escrita padrao, [10] = escrita especial, [11] = ambos
336         SinalMuxE = 2'b10;           // [00] = Switches, [01] = ULA,
        [10] = Memoria
337         SinalMuxEE = 2'b00;          // [00] = RAL, [01] = Push, [10] = Pop
338         SinalMuxPC = 2'b00;          // [00] = PC, [01] = J, [10] =
        Desvio, [11] = JR
339         SinalGEN = 3'b001;           // Num Formato - 1
340         SinalULA = 5'b01110;
341     end
342     // Formato 3 - beq

```

```

343     6'b010111:
344     begin
345         HLT = 1'b0;
346         RegULA = 1'b1;           // Registrador para ULA
347         SinalMemoria = 1'b0;     // Controla escrita
348         SinalMES = 2'b00;        // [10] = OUT, [01] = IN, [00] = default
349         SinalBanco = 2'b00;      // [00] = sem escrita, [01] =
350             escrita padrao, [10] = escrita especial, [11] = ambos
351         SinalMuxE = 2'b00;        // [00] = Switches, [01] = ULA,
352             [10] = Memoria
353         SinalMuxEE = 2'b00;       // [00] = RAL, [01] = Push, [10] = Pop
354         SinalMuxPC = 2'b10;       // [00] = PC, [01] = J, [10] =
355             Desvio, [11] = JR
356         SinalGEN = 3'b010;       // Num Formato - 1
357         SinalULA = 5'b01000;
358     end
359     // Formato 3 - bnq
360     6'b011000:
361     begin
362         HLT = 1'b0;
363         RegULA = 1'b1;           // Registrador para ULA
364         SinalMemoria = 1'b0;     // Controla escrita
365         SinalMES = 2'b00;        // [10] = OUT, [01] = IN, [00] = default
366         SinalBanco = 2'b00;      // [00] = sem escrita, [01] =
367             escrita padrao, [10] = escrita especial, [11] = ambos
368         SinalMuxE = 2'b00;        // [00] = Switches, [01] = ULA,
369             [10] = Memoria
370         SinalMuxEE = 2'b00;       // [00] = RAL, [01] = Push, [10] = Pop
371         SinalMuxPC = 2'b10;       // [00] = PC, [01] = J, [10] =
372             Desvio, [11] = JR
373         SinalGEN = 3'b010;       // Num Formato - 1
374         SinalULA = 5'b01001;
375     end
376     // Formato 3 - blt
377     6'b011001:
378     begin
379         HLT = 1'b0;
380         RegULA = 1'b1;           // Registrador para ULA
381         SinalMemoria = 1'b0;     // Controla escrita
382         SinalMES = 2'b00;        // [10] = OUT, [01] = IN, [00] = default
383         SinalBanco = 2'b00;      // [00] = sem escrita, [01] =
384             escrita padrao, [10] = escrita especial, [11] = ambos
385         SinalMuxE = 2'b00;        // [00] = Switches, [01] = ULA,
386             [10] = Memoria
387         SinalMuxEE = 2'b00;       // [00] = RAL, [01] = Push, [10] = Pop
388         SinalMuxPC = 2'b10;       // [00] = PC, [01] = J, [10] =
389             Desvio, [11] = JR
390         SinalGEN = 3'b010;       // Num Formato - 1
391         SinalULA = 5'b01010;
392     end
393     // Formato 3 - bgt
394     6'b011010:
395     begin
396         HLT = 1'b0;
397         RegULA = 1'b1;           // Registrador para ULA
398         SinalMemoria = 1'b0;     // Controla escrita
399         SinalMES = 2'b00;        // [10] = OUT, [01] = IN, [00] = default
400         SinalBanco = 2'b00;      // [00] = sem escrita, [01] =
401             escrita padrao, [10] = escrita especial, [11] = ambos

```

```

392         SinalMuxE = 2'b00;           // [00] = Switches, [01] = ULA,
           [10] = Memoria
393         SinalMuxEE = 2'b00;          // [00] = RAL, [01] = Push, [10] = Pop
394         SinalMuxPC = 2'b10;          // [00] = PC, [01] = J, [10] =
           Desvio, [11] = JR
395         SinalGEN = 3'b010;           // Num Formato - 1
396         SinalULA = 5'b01011;
397     end
398     // Formato 3 - stored
399     6'b011011:
400     begin
401         HLT = 1'b0;
402         RegULA = 1'b0;                // Registrador para ULA
403         SinalMemoria = 1'b1;          // Controla escrita
404         SinalMES = 2'b00;             // [10] = OUT, [01] = IN, [00] = default
405         SinalBanco = 2'b00;           // [00] = sem escrita, [01] =
           escrita padrao, [10] = escrita especial, [11] = ambos
406         SinalMuxE = 2'b00;           // [00] = Switches, [01] = ULA,
           [10] = Memoria
407         SinalMuxEE = 2'b00;          // [00] = RAL, [01] = Push, [10] = Pop
408         SinalMuxPC = 2'b00;          // [00] = PC, [01] = J, [10] =
           Desvio, [11] = JR
409         SinalGEN = 3'b010;           // Num Formato - 1
410         SinalULA = 5'b00000;
411     end
412     // Formato 3 - storer
413     6'b011100:
414     begin
415         HLT = 1'b0;
416         RegULA = 1'b0;                // Registrador para ULA
417         SinalMemoria = 1'b1;          // Controla escrita
418         SinalMES = 2'b00;             // [10] = OUT, [01] = IN, [00] = default
419         SinalBanco = 2'b00;           // [00] = sem escrita, [01] =
           escrita padrao, [10] = escrita especial, [11] = ambos
420         SinalMuxE = 2'b00;           // [00] = Switches, [01] = ULA,
           [10] = Memoria
421         SinalMuxEE = 2'b00;          // [00] = RAL, [01] = Push, [10] = Pop
422         SinalMuxPC = 2'b00;          // [00] = PC, [01] = J, [10] =
           Desvio, [11] = JR
423         SinalGEN = 3'b010;           // Num Formato - 1
424         SinalULA = 5'b01110;
425     end
426     // Formato 4 - movi
427     6'b011101:
428     begin
429         HLT = 1'b0;
430         RegULA = 1'b0;                // Registrador para ULA
431         SinalMemoria = 1'b0;          // Controla escrita
432         SinalMES = 2'b00;             // [10] = OUT, [01] = IN, [00] = default
433         SinalBanco = 2'b01;           // [00] = sem escrita, [01] =
           escrita padrao, [10] = escrita especial, [11] = ambos
434         SinalMuxE = 2'b01;           // [00] = Switches, [01] = ULA,
           [10] = Memoria
435         SinalMuxEE = 2'b00;          // [00] = RAL, [01] = Push, [10] = Pop
436         SinalMuxPC = 2'b00;          // [00] = PC, [01] = J, [10] =
           Desvio, [11] = JR
437         SinalGEN = 3'b011;           // Num Formato - 1
438         SinalULA = 5'b01111;
439     end
440     // Formato 4 - loadi

```

```

441      6'b011110:
442      begin
443          HLT = 1'b0;
444          RegULA = 1'b0;                // Registrador para ULA
445          SinalMemoria = 1'b0;          // Controla escrita
446          SinalMES = 2'b00;            // [10] = OUT, [01] = IN, [00] = default
447          SinalBanco = 2'b01;          // [00] = sem escrita, [01] =
448          escrita padrao, [10] = escrita especial, [11] = ambos
449          SinalMuxE = 2'b10;            // [00] = Switches, [01] = ULA,
450          [10] = Memoria
451          SinalMuxEE = 2'b00;           // [00] = RAL, [01] = Push, [10] = Pop
452          SinalMuxPC = 2'b00;           // [00] = PC, [01] = J, [10] =
453          Desvio, [11] = JR
454          SinalGEN = 3'b011;            // Num Formato - 1
455          SinalULA = 5'b01111;
456      end
457      // Formato 4 - in
458      6'b011111:
459      begin
460          HLT = 1'b0;
461          RegULA = 1'b0;                // Registrador para ULA
462          SinalMemoria = 1'b0;          // Controla escrita
463          SinalMES = 2'b01;            // [10] = OUT, [01] = IN, [00] = default
464          SinalBanco = 2'b01;          // [00] = sem escrita, [01] =
465          escrita padrao, [10] = escrita especial, [11] = ambos
466          SinalMuxE = 2'b00;            // [00] = Switches, [01] = ULA,
467          [10] = Memoria
468          SinalMuxEE = 2'b00;           // [00] = RAL, [01] = Push, [10] = Pop
469          SinalMuxPC = 2'b00;           // [00] = PC, [01] = J, [10] =
470          Desvio, [11] = JR
471          SinalGEN = 3'b011;            // Num Formato - 1
472          SinalULA = 5'b11111;
473      end
474      // Formato 4 - pop
475      6'b100000:
476      begin
477          HLT = 1'b0;
478          RegULA = 1'b0;                // Registrador para ULA
479          SinalMemoria = 1'b0;          // Controla escrita
480          SinalMES = 2'b00;            // [10] = OUT, [01] = IN, [00] = default
481          SinalBanco = 2'b11;          // [00] = sem escrita, [01] =
482          escrita padrao, [10] = escrita especial, [11] = ambos
483          SinalMuxE = 2'b10;            // [00] = Switches, [01] = ULA,
484          [10] = Memoria
485          SinalMuxEE = 2'b10;           // [00] = RAL, [01] = Push, [10] = Pop
486          SinalMuxPC = 2'b00;           // [00] = PC, [01] = J, [10] =
487          Desvio, [11] = JR
488          SinalGEN = 3'b011;            // Num Formato - 1
489          SinalULA = 5'b01110;
490      end
491      // Formato 5 - out
492      6'b100001:
493      begin
494          HLT = 1'b0;
495          RegULA = 1'b0;                // Registrador para ULA
496          SinalMemoria = 1'b0;          // Controla escrita
497          SinalMES = 2'b10;            // [10] = OUT, [01] = IN, [00] =
498          default
499          SinalBanco = 2'b00;           // [00] = sem escrita, [01] =
500          escrita padrao, [10] = escrita especial, [11] = ambos

```

```

490         SinalMuxE = 2'b00;           // [00] = Switches, [01] = ULA,
         [10] = Memoria
491         SinalMuxEE = 2'b00;          // [00] = RAL, [01] = Push, [10] = Pop
492         SinalMuxPC = 2'b00;           // [00] = PC, [01] = J, [10] =
         Desvio, [11] = JR
493         SinalGEN = 3'b100;           // Num Formato - 1
494         SinalULA = 5'b11111;
495     end
496     // Formato 5 - jr
497     6'b100010:
498     begin
499         HLT = 1'b0;
500         RegULA = 1'b0;                // Registrador para ULA
501         SinalMemoria = 1'b0;          // Controla escrita
502         SinalMES = 2'b00;             // [10] = OUT, [01] = IN, [00] = default
503         SinalBanco = 2'b00;           // [00] = sem escrita, [01] =
         escrita padrao, [10] = escrita especial, [11] = ambos
504         SinalMuxE = 2'b00;           // [00] = Switches, [01] = ULA,
         [10] = Memoria
505         SinalMuxEE = 2'b00;          // [00] = RAL, [01] = Push, [10] = Pop
506         SinalMuxPC = 2'b11;           // [00] = PC, [01] = J, [10] =
         Desvio, [11] = JR
507         SinalGEN = 3'b100;           // Num Formato - 1
508         SinalULA = 5'b11111;
509     end
510     // Formato 6 - storei
511     6'b100011:
512     begin
513         HLT = 1'b0;
514         RegULA = 1'b0;                // Registrador para ULA
515         SinalMemoria = 1'b1;          // Controla escrita
516         SinalMES = 2'b00;             // [10] = OUT, [01] = IN, [00] = default
517         SinalBanco = 2'b00;           // [00] = sem escrita, [01] =
         escrita padrao, [10] = escrita especial, [11] = ambos
518         SinalMuxE = 2'b00;           // [00] = Switches, [01] = ULA,
         [10] = Memoria
519         SinalMuxEE = 2'b00;          // [00] = RAL, [01] = Push, [10] = Pop
520         SinalMuxPC = 2'b00;           // [00] = PC, [01] = J, [10] =
         Desvio, [11] = JR
521         SinalGEN = 3'b101;           // Num Formato - 1
522         SinalULA = 5'b01111;
523     end
524     // Formato 6 - push
525     6'b100100:
526     begin
527         HLT = 1'b0;
528         RegULA = 1'b0;                // Registrador para ULA
529         SinalMemoria = 1'b1;          // Controla escrita
530         SinalMES = 2'b00;             // [10] = OUT, [01] = IN, [00] = default
531         SinalBanco = 2'b10;           // [00] = sem escrita, [01] =
         escrita padrao, [10] = escrita especial, [11] = ambos
532         SinalMuxE = 2'b00;           // [00] = Switches, [01] = ULA,
         [10] = Memoria
533         SinalMuxEE = 2'b01;          // [00] = RAL, [01] = Push, [10] = Pop
534         SinalMuxPC = 2'b00;           // [00] = PC, [01] = J, [10] =
         Desvio, [11] = JR
535         SinalGEN = 3'b101;           // Num Formato - 1
536         SinalULA = 5'b01110;
537     end
538

```

```

539      // NOVAS INSTRUÇÕES 2026
540
541      // Formato 3 - blet
542      6'b100101:
543      begin
544          HLT = 1'b0;
545          RegULA = 1'b1;                // Registrador para ULA
546          SinalMemoria = 1'b0;          // Controla escrita
547          SinalMES = 2'b00;             // [10] = OUT, [01] = IN, [00] = default
548          SinalBanco = 2'b00;           // [00] = sem escrita, [01] =
549          escrita padrao, [10] = escrita especial, [11] = ambos
550          SinalMuxE = 2'b00;             // [00] = Switches, [01] = ULA,
551          [10] = Memoria
552          SinalMuxEE = 2'b00;            // [00] = RAL, [01] = Push, [10] = Pop
553          SinalMuxPC = 2'b10;            // [00] = PC, [01] = J, [10] =
554          Desvio, [11] = JR
555          SinalGEN = 3'b010;             // Num Formato - 1
556          SinalULA = 5'b10001;
557      end
558      // Formato 3 - bget
559      6'b100110:
560      begin
561          HLT = 1'b0;
562          RegULA = 1'b1;                // Registrador para ULA
563          SinalMemoria = 1'b0;          // Controla escrita
564          SinalMES = 2'b00;             // [10] = OUT, [01] = IN, [00] = default
565          SinalBanco = 2'b00;           // [00] = sem escrita, [01] =
566          escrita padrao, [10] = escrita especial, [11] = ambos
567          SinalMuxE = 2'b00;             // [00] = Switches, [01] = ULA,
568          [10] = Memoria
569          SinalMuxEE = 2'b00;            // [00] = RAL, [01] = Push, [10] = Pop
570          SinalMuxPC = 2'b10;            // [00] = PC, [01] = J, [10] =
571          Desvio, [11] = JR
572          SinalGEN = 3'b010;             // Num Formato - 1
573          SinalULA = 5'b10010;
574      end
575      // Default - HLT
576      default:
577      begin
578          HLT = 1'b1;
579          RegULA = 1'b0;                // Registrador para ULA
580          SinalMemoria = 1'b0;          // Controla escrita
581          SinalMES = 2'b00;             // [10] = OUT, [01] = IN, [00] = default
582          SinalBanco = 2'b00;           // [00] = sem escrita, [01] =
583          escrita padrao, [10] = escrita especial, [11] = ambos
584          SinalMuxE = 2'b00;             // [00] = Switches, [01] = ULA,
585          [10] = Memoria
586          SinalMuxEE = 2'b00;            // [00] = RAL, [01] = Push, [10] = Pop
587          SinalMuxPC = 2'b00;            // [00] = PC, [01] = J, [10] =
588          Desvio, [11] = JR
589          SinalGEN = 3'b110;             // Num Formato - 1
590          SinalULA = 5'b11111;
591      end
592  endcase
593 end

```

```
587 endmodule
```

Código 4.7 – Módulo da unidade de controle em Verilog

4.6.8 Módulo de entrada e saída

No [Código 4.8](#) abaixo é apresentado o código desenvolvido para o módulo de entrada e saída.

```

1 module Modulo_Entrada_E_Saida(
2   input clock,
3
4   input [1:0] SinalMES,           // [10] = OUT, [01] = IN, [00] = default
5
6   input [31:0] Display_IN,
7
8   input [17:0] Switches_IN,
9
10  output [31:0] Switches_OUT,
11
12  input Destrava,
13  output reg TravaPC,
14
15  output [6:0] Display_0,
16  output [6:0] Display_1,
17  output [6:0] Display_2,
18  output [6:0] Display_3,
19  output [6:0] Display_4,
20  output [6:0] Display_5,
21  output [6:0] Display_6,
22  output [6:0] Display_7
23 );
24 // Para iterar no for da conversao binario para decimal
25 integer i;
26
27 // Modulo de entrada original com debounce
28 reg Controle;
29 reg [21:0] Contador;
30
31 // Codigo debounce de pulso unico
32 always @(negedge clock)
33   begin
34       if(Destrava == 1'b1) Contador <= 22'd0; // Botao solto = 1
35       else if(Contador < 22'd60000) Contador <= Contador + 1;
36
37       if(Contador == 22'd50000) Controle <= 1'b1;
38       else Controle <= 1'b0;
39   end
40
41 // Controle do TravaPC
42 always @(SinalMES[0] or Controle)
43   begin
44       if(~SinalMES[0] || Controle) TravaPC <= 1'b0;
45       else TravaPC <= 1'b1;
46   end
47
48 // Leitura e extensao dos Switches
49 assign Switches_OUT = {{14{Switches_IN[17]}}, Switches_IN[17:0]};
50
51
52 // Modulo de saida
53
54 reg [31:0] reg_out;

```

```

55 reg aux = 1'b1;
56
57 always @(negedge clock)
58 begin
59     if(aux == 1'b0)
60         begin
61             reg_out = 32'b0;
62             aux = 1'b1;
63         end
64     else
65         begin
66             if(SinalMES[1]) reg_out = Display_IN;
67             else reg_out = reg_out;
68         end
69 end
70
71
72 reg [31:0] bcd;
73
74 // Converte de Binario para BCD usando Double Dabble (Shift-and-Add-3)
75 always @(*) begin
76     bcd = 0;
77
78     // Itera por todos os bits da entrada
79     for (i = 31; i >= 0; i = i - 1) begin
80
81         // Verifica cada nibble (digito de 4 bits). Se for maior igual a 5, soma 3.
82         if (bcd[3:0] >= 5) bcd[3:0] = bcd[3:0] + 3;
83         if (bcd[7:4] >= 5) bcd[7:4] = bcd[7:4] + 3;
84         if (bcd[11:8] >= 5) bcd[11:8] = bcd[11:8] + 3;
85         if (bcd[15:12] >= 5) bcd[15:12] = bcd[15:12] + 3;
86         if (bcd[19:16] >= 5) bcd[19:16] = bcd[19:16] + 3;
87         if (bcd[23:20] >= 5) bcd[23:20] = bcd[23:20] + 3;
88         if (bcd[27:24] >= 5) bcd[27:24] = bcd[27:24] + 3;
89         if (bcd[31:28] >= 5) bcd[31:28] = bcd[31:28] + 3;
90
91         // Desloca todo o vetor BCD 1 bit para a esquerda e puxa o proximo bit do reg_out
92         bcd = {bcd[30:0], reg_out[i]};
93     end
94 end
95
96 // Usa os nibbles do registrador BCD diretamente nos decodificadores.
97 Decodificador_BCD dec0(.bcd(bcd[3:0]), .seg(Display_0));
98 Decodificador_BCD dec1(.bcd(bcd[7:4]), .seg(Display_1));
99 Decodificador_BCD dec2(.bcd(bcd[11:8]), .seg(Display_2));
100 Decodificador_BCD dec3(.bcd(bcd[15:12]), .seg(Display_3));
101 Decodificador_BCD dec4(.bcd(bcd[19:16]), .seg(Display_4));
102 Decodificador_BCD dec5(.bcd(bcd[23:20]), .seg(Display_5));
103 Decodificador_BCD dec6(.bcd(bcd[27:24]), .seg(Display_6));
104 Decodificador_BCD dec7(.bcd(bcd[31:28]), .seg(Display_7));
105
106 endmodule
107
108
109
110 module Decodificador_BCD(
111     input [3:0] bcd,
112     output reg [6:0] seg
113 );
114

```

```
115 always @(*)
116 begin
117     case(bcd)
118         4'b0000 : seg = 7'b1000000;
119         4'b0001 : seg = 7'b1111001;
120         4'b0010 : seg = 7'b0100100;
121         4'b0011 : seg = 7'b0110000;
122         4'b0100 : seg = 7'b0011001;
123         4'b0101 : seg = 7'b0010010;
124         4'b0110 : seg = 7'b0000010;
125         4'b0111 : seg = 7'b1111000;
126         4'b1000 : seg = 7'b0000000;
127         4'b1001 : seg = 7'b0011000;
128         default : seg = 7'bXXXXXXX;
129     endcase
130 end
131 endmodule
```

Código 4.8 – Módulo de entrada e saída em Verilog

4.6.9 Integração final

No [Código 4.9](#) abaixo é apresentado o código desenvolvido para a integração final.

```

1 module Processador_Completo (
2 // Inputs
3 input clock,                                // Clock do
    processador
4 input Destrava,                            // Botao para destravar o
    PC
5 input [17:0] Switches_IN,                  // Valor bruto
6
7 // Outputs
8 output [6:0] Display_0,
9 output [6:0] Display_1,
10 output [6:0] Display_2,
11 output [6:0] Display_3,
12 output [6:0] Display_4,
13 output [6:0] Display_5,
14 output [6:0] Display_6,
15 output [6:0] Display_7
16 );
17
18 // Modulo de entrada e saida
19 wire TravaPC;
20 wire [31:0] Switches;                      // Valor ja extendido //
    Entradas e saidas do banco
21 wire [5:0] reg_leitura1;                  // Endereco do registrador 1
22 wire [5:0] reg_leitura2;                  // Endereco do registrador 2
23 wire [5:0] reg_escrita;                   // Endereco de escrita
24 wire [5:0] reg_especial;                  // Endereco de escrita especial
25 wire [31:0] saida1;                       // Valor do primeiro registrador
    lido
26 wire [31:0] saida2;                       // Valor do segundo registrador
    lido
27
28 // Imediato
29 wire [31:0] Imediato;                     // Imediato
30
31 // Entradas e saidas para ULA
32 wire [31:0] SL;                           // Saida padrao
33 wire SD;                                  // Saida
    para desvio
34
35 // Atualizacao do PC
36 wire [7:0] Velho_PC;                      // PC atual
37 wire [7:0] PC;                           // PC incrementado
38 wire [7:0] Novo_PC;                       // Valor pelo qual o PC sera
    atualizado
39 wire [7:0] Desvio;                        // PC + imediato
40
41 // Escrita padrao
42 wire [31:0] saida_MEM;                    // Saida da memoria
43 wire [31:0] SE;                           // Dado escrita
44
45 // Escrita especial
46 wire [31:0] RPI_Dec;                      // Usado pelo pop
47 wire [31:0] SEE;                          // Dado especial
48
49 // Campos da instrucao

```

```

50 wire [5:0] Opcode;
51 wire [25:0] Operandos;
52
53 // Sinais de controle
54 wire HLT;
55 wire RegULA; // Registrador
    para ULA
56 wire SinalMemoria; // Controla escrita
57 wire [1:0] SinalMES; // [10] = OUT, [01] = IN, [00] =
    default
58 wire [1:0] SinalBanco; // [00] = sem escrita, [01] = escrita
    padrao, [10] = escrita especial, [11] = ambos
59 wire [1:0] SinalMuxE; // [00] = Switches, [01] = ULA,
    [10] = Memoria
60 wire [1:0] SinalMuxEE; // [00] = RAL, [01] = Push, [10] = Pop
61 wire [1:0] SinalMuxPC; // [00] = PC, [01] = J, [10] = Desvio,
    [11] = JR
62 wire [2:0] SinalGEN; // Numero do formato menos 1
63 wire [4:0] SinalULA; //Codigo de operacao da ULA
64
65 // Conexoes com a ULA
66 wire [31:0] EA;
67 wire [31:0] EB;
68
69 reg [24:0] Contador;
70 reg clock_slow;
71
72 always @(posedge clock)
73     begin
74         if(Contador < 23'd49) Contador <= Contador + 1'b1;
75         else Contador <= 23'd0;
76
77         if(Contador > 23'd24) clock_slow <= 1'b1;
78         else clock_slow <= 1'b0;
79     end
80
81 Unidade_de_Controlo uc_inst(
82     .Opcode(Opcode),
83     .HLT(HLT),
84     .RegULA(RegULA),
85     .SinalMemoria(SinalMemoria),
86     .SinalMES(SinalMES),
87     .SinalBanco(SinalBanco),
88     .SinalMuxE(SinalMuxE),
89     .SinalMuxEE(SinalMuxEE),
90     .SinalMuxPC(SinalMuxPC),
91     .SinalGEN(SinalGEN),
92     .SinalULA(SinalULA)
93 );
94
95 // Atualizacao do PC
96 PC pc_inst(
97     .clock(clock_slow),
98     .Novo_Endereco(Novo_PC), // Endereco novo
99     .HLT(HLT),
100     .TravaPC(TravaPC),
101     .PC(Velho_PC) // Passa a ser o endereco
        "velho"
102 );
103

```

```

104 Memoria_Instrucoes mem_inst(
105     .clock(clock_slow),
106     .PC(Velho_PC),
107     .Opcode(Opcode),
108     .Operandos(Operandos)
109 );
110
111 Gerenciador_de_Entrada ger_ent(
112     .SinalGEN(SinalGEN),
113     .Operandos(Operandos),
114     .L1(reg_leitura1),
115     .L2(reg_leitura2),
116     .E1(reg_escrita),
117     .E2(reg_especial),
118     .Imediato(Imediato)
119 );
120
121 // Incrementa o PC
122 Add1 add1_inst(
123     .entrada(Velho_PC),
124     .saida(PC)
125 );
126
127 // Calcula o desvio
128 Adder adder_inst(
129     .entrada1(PC),
130     .entrada2(Imediato[7:0]),
131     .saida(Desvio)
132 );
133
134 // Mux do PC
135 Mux_PC mux_pc_inst(
136     .SinalMuxPC(SinalMuxPC),          // Sinal de controle
137     .SD(SD),
138     .PC(PC),                          // Caso va fazer
139     .J(Imediato[7:0]),                // Usa o imediato para jmp (8 bits LSB)
140     .Desvio(Desvio),                  // Desvio calculado com PC+1+
141     .JR(saida1[7:0]),                 // Primeiro registrador lido (8
142     .bits LSB)                        bits LSB)
143     .Novo_PC(Novo_PC)                 // Valor para atualizar o PC
144 );
145
146 // Envia os dados para o banco
147 Banco_de_Registradores #(.ADDR_WIDTH(4)) banco_inst(
148     .clock(clock_slow),
149     .reg_leitura1(reg_leitura1),
150     .reg_leitura2(reg_leitura2),
151     .reg_escrita(reg_escrita),
152     .reg_especial(reg_especial),
153     .dado_escrita(SE),                // Saida do mux de
154     .escrita
155     .dado_especial(SEE),              // Saida do mux de escrita
156     .especial
157     .SinalBanco(SinalBanco),
158     .saida1(saida1),
159     .saida2(saida2)
160 );

```

```

159 // Atribui os valores de entrada para ULA
160 // Primeira entrada da ULA vem da saida1 do banco
161 assign EA = saida1;
162
163 // Segunda entrada vem ou da saida2 ou do imediato
164 Mux2 mux_ula_in(
165     .E1(saida2),
166     .E2(Imediato),
167     .S(EB),
168     .Sinal(RegULA)
169 );
170
171 // Envia os dados para ULA
172 ULA ula_inst(
173     .EA(EA),
174     .EB(EB),
175     .SinalULA(SinalULA),
176     .SL(SL),
177     .SD(SD)
178 );
179
180 // Decrementa o endereco da pilha
181 Sub1 sub1_inst(
182     .entrada(SL),
183     .saida(RPI_Dec)
184 );
185
186 // Mux de escrita especial
187 Mux_Escrita_Especial mux_esc_esc(
188     .SinalMuxEE(SinalMuxEE),
189     .RAL(PC), // Usado no JAL
190     .Push(SL), // Operacoes push
191     .RPI_incrementa
192     .Pop(RPI_Dec), // Operacoes pop. RPI
193     .Decrementa
194     .SEE(SEE)
195 );
196
197 // Memoria principal
198 Memoria_Principal mem_prin(
199     .clock(clock_slow),
200     .END(SL), // Saida da ULA -
201     .Seleciona_apenas_os_menos_significativos
202     .dado_escrita(saida2), // Segunda leitura do banco
203     .SinalMemoria(SinalMemoria),
204     .saida(saida_MEM)
205 );
206
207 // Mux de escrita
208 Mux_Escrita mux_esc(
209     .SinalMuxE(SinalMuxE),
210     .Switches(Switches),
211     .ULA(SL),
212     .Memoria(saida_MEM),
213     .SE(SE)
214 );
215
216 // Modulo de entrada e saida
217 Modulo_Entrada_E_Saida mes_inst(
218     .clock(clock_slow),

```

```
216     .SinalMES(SinalMES),  
217     .Display_IN(saida1),  
218     .Switches_IN(Switches_IN),  
219     .Switches_OUT(Switches),  
220     .Destrava(Destrava),  
221     .TravaPC(TravaPC),  
222     .Display_0(Display_0),  
223     .Display_1(Display_1),  
224     .Display_2(Display_2),  
225     .Display_3(Display_3),  
226     .Display_4(Display_4),  
227     .Display_5(Display_5),  
228     .Display_6(Display_6),  
229     .Display_7(Display_7)  
230 );  
231  
232 endmodule
```

Código 4.9 – Integração final em Verilog

5 Desenvolvimento do Compilador

5.1 Fase de Análise

A fase de análise é o *front-end* do compilador, ou seja, a parte responsável por ler o código-fonte em C-, entender a sua estrutura e verificar se ele está bem escrito. Ao final dessa fase, o programa deixa de ser um simples texto e passa a ser representado internamente por uma árvore sintática abstrata já validada, que serve de base para a fase de síntese. Essa fase é dividida em três etapas: a análise léxica, a análise sintática e a análise semântica (LOUDEN, 2004).

No projeto, a análise léxica foi construída com o auxílio da ferramenta Flex (Free Software Foundation, 2012), que gera o analisador léxico a partir de um conjunto de expressões regulares, enquanto a análise sintática foi construída com o Bison (Free Software Foundation, 2021), que gera um analisador sintático do tipo LALR a partir da gramática da linguagem. À medida que o analisador sintático reconhece a estrutura do programa, ele vai construindo a árvore sintática abstrata, que depois é percorrida pela análise semântica para montar a tabela de símbolos e verificar os tipos.

5.1.1 Modelagem

Antes da implementação, foi feita a modelagem da fase de análise utilizando a linguagem SysML, com o objetivo de organizar as ideias e representar tanto a estrutura quanto o comportamento dessa etapa do compilador.

5.1.1.1 Diagrama de Blocos (SysML)

Observação: o diagrama de blocos SysML referente à fase de análise não pôde ser incluído neste relatório. Devido a um problema pessoal com o meu computador, acabei perdendo os arquivos originais dos diagramas do compilador, não sendo possível recuperá-los a tempo da entrega. O conteúdo que esse diagrama representava está descrito ao longo das seções seguintes deste capítulo.

5.1.1.2 Diagramas de Atividades (SysML)

Observação: os diagramas de atividades SysML referentes à fase de análise também foram perdidos junto com os demais arquivos, pelo mesmo problema pessoal com o computador. O comportamento que esses diagramas descreviam,

ou seja, o fluxo de reconhecimento dos tokens, a construção da árvore e as verificações semânticas, está explicado em texto nas seções a seguir.

5.1.1.3 Outros: a Árvore Sintática Abstrata

Além dos diagramas, uma estrutura central da fase de análise é a árvore sintática abstrata (AST). Ela representa o programa em forma de árvore, na qual cada nó corresponde a uma construção da linguagem, como uma declaração, uma expressão ou um comando. A definição do nó da árvore é apresentada no Código 5.1, onde é possível observar o tipo do nó, o tipo da expressão, os ponteiros para os filhos e para o próximo nó da lista, além dos campos usados para guardar números e identificadores.

```

1  typedef enum {
2      NODE_VAR_DECL,      NODE_FUN_DECL,      NODE_COMPOUND_STMT,
3      NODE_IF_STMT,      NODE_WHILE_STMT,     NODE_RETURN_STMT,
4      NODE_ASSIGN_EXPR,  NODE_BINARY_OP,     NODE_NUM,
5      NODE_VAR,          NODE_FUN_CALL,     NODE_ARRAY_DECL,
6      NODE_ARRAY_ACCESS, NODE_PARAM,        NODE_PARAM_LIST,
7      NODE_TYPE,         NODE_OPERATOR,     NODE_FUN_BODY
8  } NodeType;
9
10 typedef enum { Void, Integer, Boolean } ExpType;
11
12 typedef struct ASTNode {
13     NodeType type;
14     ExpType expType;
15     struct ASTNode *leftChild;
16     struct ASTNode *rightChild;
17     struct ASTNode *next;
18     int number;
19     char* identifier;
20     int lineno;
21 } ASTNode;

```

Código 5.1 – Definição do nó da árvore sintática abstrata (ast.h)

A escolha de usar um único tipo de nó com campos genéricos, ao invés de um tipo diferente para cada construção, deixa a árvore mais simples de percorrer, já que a fase de síntese pode tratar todos os nós de maneira uniforme, olhando apenas para o campo que indica o tipo do nó.

5.1.2 Análise Léxica

A análise léxica é a primeira etapa do compilador. Ela recebe o código-fonte como uma sequência de caracteres e agrupa esses caracteres em unidades com significado, chamadas de tokens, como palavras reservadas, identificadores, números e operadores. Ao mesmo tempo, ela descarta espaços em branco e comentários, e reporta os caracteres que não pertencem à linguagem.

O analisador léxico foi descrito no arquivo `scanner.1` utilizando o Flex. Cada regra associa uma expressão regular a uma ação, que normalmente devolve o token correspondente para o analisador sintático. O Código 5.2 mostra um trecho dessas regras, incluindo o tratamento das palavras reservadas, dos operadores, dos números, dos identificadores e dos comentários no formato `/* */`.

```

1  digito      [0-9]
2  letra      [a-zA-Z]
3  numero     {digito}+
4  identificador {letra}({letra}|{digito})*
5  whitespace [ \t]+
6  quebralinha [\r\n]+
7
8  %%
9
10 "/*"      { char c1 = 0;
11              char c2 = 0;
12              do
13              { c1 = c2;
14                c2 = input();
15                if (c2 == EOF) break;
16                if (c2 == '\n') yylineno++;
17              } while (c1 != '*' || c2 != '/');
18              }
19
20 "+"        { return ADD; }
21 "-"        { return SUB; }
22 "*"        { return MUL; }
23 "/"        { return DIV; }
24
25 "<="        { return LET; }
26 "<"         { return LT;  }
27 ">="        { return GET; }
28 ">"         { return GT;  }
29 "=="        { return EQ;  }
30 "!="        { return DIF; }
31
32 "="         { return ASSIGN; }
33 ";"         { return SEMICOLON; }
34 ",",        { return COMMA; }
35
36 "("         { return OPENPAR; }
37 ")"         { return CLOSEPAR; }
38 "["         { return OPENCOL; }
39 "]"         { return CLOSECOL; }
40 "{"         { return OPENCHA; }
41 "}"         { return CLOSECHA; }
42
43 "if"        { return IF; }
44 "else"      { return ELSE; }
45 "int"       { return INT; }
46 "return"    { return RETURN; }
47 "void"      { return VOID; }
48 "while"     { return WHILE; }
49

```

```

50 {numero}      { yylval.ival = atoi(yytext); return NUM; }
51 {identificador} { yylval.sval = strdup(yytext); return ID; }
52
53 {whitespace}  { }
54 \n            { yylineno++; }
55
56 .             { printf("[Lexical Error] Line %d: Unrecognized character '%s'\n",
                        yylineno, yytext); return ERRO; }

```

Código 5.2 – Trecho das regras léxicas (scanner.l)

Para os números, a ação converte o texto reconhecido em um valor inteiro com a função `atoi`, e para os identificadores é feita uma cópia do nome com `strdup`, de modo que esse nome possa ser usado depois na tabela de símbolos. Além disso, a cada token reconhecido, a linha atual é guardada por meio da macro `YY_USER_ACTION`, o que permite que os erros sejam reportados junto com o número da linha onde ocorreram. Quando o analisador encontra um caractere que não faz parte da linguagem, ele emite uma mensagem de erro léxico.

5.1.3 Análise Sintática

A análise sintática recebe a sequência de tokens produzida pela análise léxica e verifica se ela respeita a gramática da linguagem C-. Se a estrutura estiver correta, o analisador constrói a árvore sintática abstrata que representa o programa. Se não estiver, ele reporta um erro sintático.

Esse analisador foi descrito no arquivo `parser.y` com o Bison, que gera um analisador do tipo LALR. Um ponto importante na definição da gramática é a declaração de precedência e associatividade dos operadores, mostrada no Código 5.3. Nela, os operadores de soma e subtração recebem uma precedência menor do que os de multiplicação e divisão, o que garante que uma expressão como `a + b * c` seja interpretada da maneira esperada.

```

1 %start programa
2
3 %left ADD SUB
4 %left MUL DIV
5
6 %nonassoc IFX
7 %nonassoc ELSE
8
9 %define parse.error verbose
10 %locations
11
12 %token ERRO
13 %token ADD SUB MUL DIV
14 %token LT LET GET GT EQ DIF
15 %token ASSIGN SEMICOLON COMMA
16 %token OPENPAR CLOSEPAR OPENCOL CLOSECOL OPENCHA CLOSECHA
17 %token IF ELSE INT RETURN VOID WHILE
18

```

```

19 %token <ival> NUM
20 %token <sval> ID
21
22 %type <node> programa declaracao_lista declaracao var_declaracao
      tipo_especificador fun_declaracao
23 %type <node> params param_lista param composto_decl local_declaracoes
      statement_lista statement
24 %type <node> expressao_decl selecao_decl iteracao_decl retorno_decl expressao
      var simples_expressao
25 %type <node> relacional soma_expressao termo fator ativacao args arg_lista

```

Código 5.3 – Precedência e associatividade dos operadores (parser.y)

Ainda sobre a precedência, foi utilizado o recurso de precedência para resolver a ambiguidade do *dangling else*, ou seja, o problema de decidir a qual `if` um `else` pertence quando existem estruturas de decisão aninhadas. O Código 5.4 apresenta um trecho das regras da gramática, onde é possível observar como as produções são associadas às ações que constroem a árvore. Cada vez que uma regra é reconhecida, um nó é criado e ligado aos seus filhos, montando a árvore de baixo para cima.

```

1  var_declaracao: tipo_especificador ID SEMICOLON
2      {
3          ASTNode* id_node = create_leaf_id($2);
4          id_node->lineno = @2.first_line;
5          $$ = create_node(NODE_VAR_DECL, $1, id_node);
6          $$->lineno = @2.first_line;
7      }
8      | tipo_especificador ID OPENCOL NUM CLOSECOL SEMICOLON
9      {
10         ASTNode* id_node = create_leaf_id($2);
11         id_node->lineno = @2.first_line;
12         ASTNode* num_node = create_leaf_num($4);
13         num_node->lineno = @4.first_line;
14         ASTNode* array_decl = create_node(NODE_ARRAY_DECL, id_node,
15             num_node);
16         $$ = create_node(NODE_VAR_DECL, $1, array_decl);
17     }
18     | error SEMICOLON
19     { yyerrok; $$ = NULL; }
20
21  tipo_especificador: INT
22      { $$ = create_leaf_type(INT); }
23      | VOID
24      { $$ = create_leaf_type(VOID); }
25
26
27  fun_declaracao: tipo_especificador ID OPENPAR params CLOSEPAR composto_decl
28      {
29          ASTNode* id_node = create_leaf_id($2);
30          id_node->lineno = @2.first_line;
31          ASTNode* func_node = create_node(NODE_FUN_DECL, $1, id_node);
32          func_node->lineno = @2.first_line;
33          ASTNode* body_node = create_node(NODE_FUN_BODY, $4, $6);
34          func_node->next = body_node;

```

```
35         $$ = func_node;  
36     }  
37     ;
```

Código 5.4 – Trecho da gramática e construção da árvore (parser.y)

Também foi adicionada uma forma simples de recuperação de erros, por meio de regras que usam o símbolo especial **error** seguido de **yyerrok**. Com isso, ao encontrar um erro em um comando, o analisador consegue descartar o trecho problemático e continuar a análise a partir do próximo ponto-e-vírgula, ao invés de parar na primeira falha.

5.1.4 Análise Semântica

A análise semântica é a etapa que verifica o significado do programa, indo além da sua estrutura. Nesse ponto, o compilador já sabe que o programa está bem formado do ponto de vista sintático, mas ainda precisa checar se ele faz sentido, como, por exemplo, se as variáveis foram declaradas antes de serem usadas, se as funções são chamadas com o número certo de argumentos e se os tipos são compatíveis.

Para isso, foi construída uma tabela de símbolos, responsável por guardar as informações de cada identificador do programa, como o seu tipo, a sua categoria, o escopo ao qual pertence e as linhas em que aparece. Essa tabela foi implementada como uma tabela *hash*, cuja função de espalhamento é apresentada no Código 5.5.

```
1 // funcao hash simples  
2 static int hash ( char * key )  
3 {  
4     int temp = 0;  
5     int i = 0;  
6     while (key[i] != '\0')  
7     {  
8         temp = ((temp << SHIFT) + key[i]) % SIZE;  
9         ++i;  
10    }  
11    return temp;  
12 }
```

Código 5.5 – Função hash da tabela de símbolos (syntab.c)

A tabela de símbolos também trata os diferentes escopos do programa. Existe um escopo global, chamado de **Global**, e um escopo para cada função. A busca de um identificador é feita primeiro no escopo atual e, caso ele não seja encontrado, a busca continua no escopo global, o que permite que uma função enxergue tanto as suas próprias variáveis quanto as variáveis globais. O Código 5.6 mostra a inserção de um símbolo na tabela, considerando o escopo atual.

```
1 void st_insert( char * name, int lineno, int loc, ExpType type, IdKind kind, int  
    isArray, int numParams, ExpType *paramTypes )  
2 {  
3     int h = hash(name);
```

```

4  BucketList l = hashTable[h];
5
6  initScopeStack();
7  char * currentScopeName = scopeNameStack[scopeStackTop-1];
8
9  while ((l != NULL) && ((strcmp(name,l->name) != 0) || (strcmp(l->scopeName,
10     currentScopeName) != 0)))
11     l = l->next;
12
13  if (l == NULL)
14  {
15     l = (BucketList) malloc(sizeof(struct BucketListRec));
16     l->name = name;
17     l->lines = (LineList) malloc(sizeof(struct LineListRec));
18     l->lines->lineno = lineno;
19     l->lines->next = NULL;
20     l->memloc = loc;
21     l->type = type;
22     l->kind = kind;
23     l->isArray = isArray;
24     l->numParams = numParams;
25
26     if (paramTypes != NULL) {
27         for (int i = 0; i < numParams && i < MAX_PARAMS; i++) {
28             l->paramTypes[i] = paramTypes[i];
29         }
30     }
31
32     l->scopeName = currentScopeName;
33     l->next = hashTable[h];
34     hashTable[h] = l;
35 }

```

Código 5.6 – Inserção de um símbolo na tabela (symtab.c)

A verificação semântica é feita percorrendo a árvore sintática abstrata em duas etapas. Na primeira, a árvore é percorrida para preencher a tabela de símbolos, inserindo as declarações de variáveis e de funções, além das funções predefinidas `input` e `output`. Na segunda, a árvore é percorrida novamente para checar os tipos e reportar os erros semânticos. Entre as verificações feitas, estão a declaração de variáveis ou parâmetros do tipo `void`, a redeclaração de um mesmo identificador no mesmo escopo, o uso de identificadores não declarados, a chamada de funções com número ou tipo de argumentos incorretos, o uso indevido de vetores e a compatibilidade entre o tipo de retorno da função e o valor retornado. O Código 5.7 mostra parte dessas verificações, incluindo a checagem dos tipos dos parâmetros em uma chamada de função e a checagem do tipo de retorno.

```

1  case NODE_VAR:
2      if (st_lookup(t->identifier) == -1)
3          t->expType = Integer;
4      else
5          t->expType = st_lookup_type(t->identifier);
6      break;
7

```

```

8      case NODE_FUN_CALL:
9          if (t->leftChild != NULL && t->leftChild->type == NODE_VAR) {
10             t->expType = st_lookup_type(t->leftChild->identifier);
11
12             int expectedParams = st_lookup_num_params(t->leftChild->identifier);
13             ASTNode *arg = t->rightChild;
14             int i = 0;
15             while (arg != NULL && i < expectedParams) {
16                 ExpType expectedType = st_lookup_param_type(t->leftChild->
17                     identifier, i);
18                 if (arg->expType != expectedType) {
19                     char buf[256];
20                     sprintf(buf, "Invalid parameter type in call to '%s'", t->
21                         leftChild->identifier);
22                     typeError(t, buf);
23                 }
24                 arg = arg->next;
25                 i++;
26             }
27             break;
28
29         case NODE_ARRAY_ACCESS:
30             t->expType = Integer;
31             break;
32
33         case NODE_RETURN_STMT:
34             {
35                 ExpType funcType = st_lookup_type(currentFuncName);
36
37                 if (funcType == Void) {
38                     if (t->leftChild != NULL) {
39                         typeError(t, "Void function cannot return a value");
40                     }
41                 } else {
42                     if (t->leftChild == NULL) {
43                         typeError(t, "Function expected to return a value");
44                     } else if (t->leftChild->expType != funcType) {
45                         typeError(t, "Invalid return type");
46                     }
47                 }
48             }
49             break;
50
51         default:
52             break;
53     }

```

Código 5.7 – Trecho da verificação de tipos (analyze.c)

Por fim, ainda é verificado se a função `main` foi declarada, já que ela é o ponto de entrada do programa. Caso alguma dessas verificações falhe, o compilador reporta o erro correspondente junto com a linha em que ele ocorreu, e a compilação não avança para a fase de síntese.

5.2 Fase de Síntese

A fase de síntese é o *back-end* do compilador. Ela parte da árvore sintática abstrata já validada pela fase de análise e produz o código de saída, passando por três representações cada vez mais próximas do hardware: o código intermediário, o código assembly e o código binário. Além disso, essa fase também é responsável por decidir como as variáveis e os valores temporários serão guardados na memória e nos registradores do processador.

5.2.1 Modelagem

Assim como na fase de análise, a fase de síntese também foi modelada em SysML antes da implementação, buscando representar a estrutura das etapas de geração de código e o comportamento de cada uma delas.

5.2.1.1 Diagrama de Blocos (SysML)

Observação: o diagrama de blocos SysML referente à fase de síntese não pôde ser incluído neste relatório. Assim como os demais diagramas do compilador, ele foi perdido devido a um problema pessoal com o meu computador, não sendo possível recuperá-lo a tempo da entrega. A estrutura que esse diagrama representava está descrita ao longo deste capítulo.

5.2.1.2 Diagramas de Atividades (SysML)

Observação: os diagramas de atividades SysML referentes à fase de síntese também foram perdidos pelo mesmo motivo. O fluxo de geração do código intermediário, do assembly e do binário que esses diagramas descreviam está explicado em texto nas seções seguintes.

5.2.2 Geração de Código Intermediário

O código intermediário é uma representação que fica entre a linguagem de alto nível e o código de máquina (AHO et al., 2008). Ele é independente dos detalhes do processador, o que facilita tanto a geração do código quanto a sua leitura durante os testes. No projeto, essa representação foi feita por meio de quádruplas, que são instruções compostas por uma operação e três endereços.

O Código 5.8 apresenta os tipos usados na representação intermediária. A estrutura **Quad** guarda a operação e os três endereços, além de um ponteiro para a próxima quádrupla, formando uma lista encadeada. Cada endereço, representado pela estrutura **Address**, possui um campo que indica o seu tipo, que pode ser uma constante inteira, uma variável, um temporário, um rótulo ou um registrador, além do seu valor e do seu nome.

```
1 typedef enum {
2     OP_ADD ,
3     OP_ADDI ,
4     OP_ADDR ,
5     OP_SUB ,
6     OP_SUBI ,
7     OP_SUBR ,
8     OP_MUL ,
9     OP_MULI ,
10    OP_MULR ,
11    OP_DIV ,
12    OP_DIVI ,
13    OP_DIVR ,
14    OP_BLT ,
15    OP_BLE ,
16    OP_BGT ,
17    OP_BGE ,
18    OP_BEQ ,
19    OP_BNE ,
20    OP_IFF ,
21    OP_JUMP ,
22    OP_JR ,
23    OP_LABEL ,
24    OP_IN ,
25    OP_OUT ,
26    OP_PARAM ,
27    OP_CALL ,
28    OP_RET ,
29    OP_HALT ,
30    OP_FUN ,
31    OP_END_FUN ,
32    OP_ARG ,
33    OP_LOAD ,
34    OP_LOADDI ,
35    OP_STORE ,
36    OP_STOREDI ,
37    OP_MOV ,
38    OP_MOVI ,
39    OP_MOVR ,
40    OP_ALLOC ,
41    OP_FREE ,
42    NONE_OP
43 } QuadOp;
44
45 typedef enum {
46     EMPTY ,
47     INT_CONST ,
48     STRING_VAR ,
49     TEMP_VAR ,
50     LABEL_KIND ,
51     REGISTER_KIND
52 } OperandKind;
53
54 typedef struct {
55     OperandKind kind;
56     int val;
57     char *name;
```

```

58 } Address;
59
60 typedef struct Quad {
61     QuadOp op;
62     Address addr1;
63     Address addr2;
64     Address addr3;
65     struct Quad *next;
66 } Quad;

```

Código 5.8 – Tipos da representação intermediária (cintgen.h)

As operações possíveis nas quádruplas cobrem as principais construções da linguagem. A Tabela 18 organiza esses tipos de quádrupla em categorias, mostrando o papel de cada grupo na representação intermediária.

Tabela 18 – Tipos de quádrupla gerados pelo compilador

Categoria	Operações
Aritméticas	add, sub, mul, div (e suas variantes com imediato e registrador)
Desvios condicionais	blt, ble, bgt, bge, beq, bne
Controle de fluxo	iff, jmp, jr, lab
Funções	fun, end, arg, param, call, ret
Memória e cópia	alloc, free, load, loaddi, store, storedi, mov, movi, movr
Entrada e saída	in, out
Encerramento	halt

As quádruplas são criadas por meio da função `makeNewQuad`, mostrada no Código 5.9. Essa função aloca uma nova quádrupla, preenche a operação e os endereços e a adiciona ao final da lista encadeada. Além disso, sempre que um dos endereços é um temporário, ela atualiza uma lista de controle que acompanha o uso dos temporários, informação que é importante depois, na alocação de registradores.

```

1 Quad* makeNewQuad(QuadOp op, Address a1, Address a2, Address a3) {
2     Quad *q = (Quad *)malloc(sizeof(Quad));
3     q->op = op;
4     q->addr1 = a1;
5     q->addr2 = a2;
6     q->addr3 = a3;
7
8     // Adiciona na lista de controle
9     if(a1.kind == TEMP_VAR) update_or_insert(a1);
10    if(a2.kind == TEMP_VAR) update_or_insert(a2); // Provavelmente nem precisa
11    if(a3.kind == TEMP_VAR) update_or_insert(a3);
12
13    q->next = NULL;
14
15    if (head == NULL) {
16        head = tail = q;
17    } else {
18        tail->next = q;
19        tail = q;

```

```
20     }
21
22     return q;
23 }
```

Código 5.9 – Criação de uma quádrupla (cintgen.c)

A geração do código intermediário é feita percorrendo a árvore sintática abstrata por meio de uma grande estrutura de decisão, na qual cada tipo de nó gera as quádruplas correspondentes. O Código 5.10 mostra o tratamento de uma estrutura de decisão `if/else`, no qual são criados os rótulos para o desvio e para o fim da estrutura, e são gerados os saltos que ligam o teste da condição, o bloco do `if` e o bloco do `else`.

```
1  case NODE_IF_STMT:
2      // Verifica a condição
3
4      Address label_else = generateCode(node->leftChild, scope, 1);
5      Address label_end = *createLabelAddr();
6
7      // If
8      generateCode(node->rightChild, scope, 1);
9
10     // Fim do If
11     makeNewQuad(OP_JUMP, label_end, createEmptyAddr(), createEmptyAddr());
12
13     // Else
14     makeNewQuad(OP_LABEL, label_else, createEmptyAddr(), createEmptyAddr());
15     if(node->number == 1){
16         generateCode(node->next, scope, 1);
17     }
18
19     // Fim do else
20     makeNewQuad(OP_LABEL, label_end, createEmptyAddr(), createEmptyAddr());
21
22     break;
```

Código 5.10 – Geração de código para o `if/else` (cintgen.c)

5.2.3 Geração de Código Assembly

A geração de código assembly transforma as quádruplas em instruções da linguagem de montagem do processador. Nessa etapa, dois problemas principais precisam ser resolvidos: decidir em quais registradores os valores serão guardados e traduzir cada quádrupla para a instrução correta.

O processador possui um número limitado de registradores de uso geral, então a alocação de registradores precisa reaproveitá-los ao longo do programa. O Código 5.11 mostra a função responsável por essa alocação. Ela primeiro verifica se o temporário já está em algum registrador e, caso esteja, apenas o reutiliza. Caso contrário, ela procura um registrador livre, identificado por uma soma de usos igual a zero, e o associa ao novo temporário.

```

1 Address* allocate_register(char* temp_name, tempControl* tempControlHead,
2   regControl* regVector) {
3     if (temp_name == NULL || regVector == NULL) {
4       perror("Error: Invalid arguments passed to allocate_register");
5       return NULL;
6     }
7     // 1. Procurar se em algum dos registradores o labelName é o mesmo do temporário
8     for (int i = 0; i < NUMBER_OF_REGISTERS; i++) {
9       if (regVector[i].labelName != NULL) {
10        if (strcmp(regVector[i].labelName, temp_name) == 0) {
11          // 1.1. Caso seja, reduz em 1 o valor da soma
12          regVector[i].sum -= 1;
13
14          // Retorna o endereço do registrador encontrado
15          return &(regVector[i].reg);
16        }
17      }
18    }
19
20    // 2. Caso não tenham o labelName correspondente
21
22    // Busca o temporário no tempControl
23    tempControl* current_temp = tempControlHead;
24    int temp_sum = 1;
25    int temp_safe = 0;
26
27    while (current_temp != NULL) {
28      if (current_temp->temp.name != NULL && strcmp(current_temp->temp.name,
29        temp_name) == 0) {
30        temp_sum = current_temp->sum;
31        temp_safe = current_temp->safe;
32        break;
33      }
34      current_temp = current_temp->next;
35    }
36
37    // 2.1. Procurar o primeiro registrador cuja soma é zero
38    for (int i = 0; i < NUMBER_OF_REGISTERS; i++) {
39      if (regVector[i].sum == 0) {
40        // Se o labelName antigo existia (está sendo substituído), limpa a
41        // memória dele
42        if (regVector[i].labelName != NULL) {
43          free(regVector[i].labelName);
44        }
45
46        // Atualiza o labelName e o safe
47        regVector[i].labelName = strdup(temp_name);
48        regVector[i].safe = temp_safe;
49
50        // Inicializa a nova soma como sum(label) - 1 (pois essa já é a
51        // primeira ocorrência do temp)
52        regVector[i].sum = temp_sum - 1;
53
54        // Retorna o endereço do registrador escolhido
55        return &(regVector[i].reg);
56      }
57    }
58  }

```

```

53     }
54 }

```

Código 5.11 – Alocação de registradores (casm.c)

Outro ponto importante dessa etapa é que algumas operações da representação intermediária são genéricas e precisam ser especializadas de acordo com o tipo dos seus operandos. O Código 5.12 mostra essa conversão para as operações aritméticas. Uma operação de soma, por exemplo, é transformada em uma instrução com imediato quando o terceiro operando é uma constante, ou em uma instrução entre registradores quando o operando é um temporário ou um registrador.

```

1     case OP_ADD:
2         if(current->addr3.kind == INT_CONST) current->op = OP_ADDI;
3         else if(current->addr3.kind == TEMP_VAR || current->addr3.kind ==
4             REGISTER_KIND) current->op = OP_ADDR;
5         break;
6     case OP_SUB:
7         if(current->addr3.kind == INT_CONST) current->op = OP_SUBI;
8         else if(current->addr3.kind == TEMP_VAR || current->addr3.kind ==
9             REGISTER_KIND) current->op = OP_SUBR;
10        break;
11    case OP_DIV:
12        if(current->addr3.kind == INT_CONST) current->op = OP_DIVI;
13        else if(current->addr3.kind == TEMP_VAR || current->addr3.kind ==
14            REGISTER_KIND) current->op = OP_DIVR;
15        break;
16    case OP_MUL:
17        if(current->addr3.kind == INT_CONST) current->op = OP_MULI;
18        else if(current->addr3.kind == TEMP_VAR || current->addr3.kind ==
19            REGISTER_KIND) current->op = OP_MULR;
20        break;

```

Código 5.12 – Especialização das operações aritméticas (casm.c)

5.2.4 Geração de Código Executável

A última etapa da fase de síntese é a geração do código executável, ou seja, o código binário que o processador consegue interpretar diretamente. Cada instrução assembly é convertida em uma palavra de 32 bits, na qual os primeiros 6 bits representam o *opcode*, que identifica a operação, e os bits restantes representam os operandos, de acordo com o formato da instrução.

O Código 5.13 mostra a função que associa cada operação ao seu *opcode* de 6 bits. Esses valores seguem exatamente os códigos definidos no processador desenvolvido anteriormente, garantindo que o binário gerado pelo compilador seja compatível com o hardware.

```

1 char* getOpcodeBinary(QuadOp op) {
2     switch(op) {
3         case OP_HALT:     return "000000";

```

```

4         case OP_JUMP:      return "000010";
5         case OP_ADDR:      return "000100";
6         case OP_SUBR:      return "000101";
7         case OP_MULR:      return "000110";
8         case OP_DIVR:      return "000111";
9         case OP_ADDI:      return "001011";
10        case OP_SUBI:      return "001100";
11        case OP_MULI:      return "001101";
12        case OP_DIVI:      return "001110";
13        case OP_MOVR:      return "010100";
14        case OP_LOADDI:    return "010101";
15        case OP_BEQ:       return "010111";
16        case OP_BNE:       return "011000";
17        case OP_BLT:       return "011001";
18        case OP_BGT:       return "011010";
19        case OP_STOREDI:   return "011011";
20        case OP_MOVI:      return "011101";
21        case OP_IN:        return "011111";
22        case OP_OUT:       return "100001";
23        case OP_JR:        return "100010";
24        case OP_BLE:       return "100101";
25        case OP_BGE:       return "100110";
26        default:          return "000001"; // NOP
27    }
28 }

```

Código 5.13 – Opcodes das instruções (binary.c)

A montagem final da palavra de 32 bits leva em conta o formato de cada instrução, distribuindo os bits entre os registradores e os valores imediatos. Um cuidado especial é necessário nos desvios, cujo destino é calculado como um deslocamento relativo em relação à próxima instrução, ou seja, o rótulo de destino menos a posição do PC somada de um. O Código 5.14 mostra parte dessa montagem.

```

1 void generateBinary(Quad* quadHead, labelControl* labelHead) {
2     FILE* file = fopen("output/binary.txt", "w");
3     if (!file) return;
4
5     Quad* current = quadHead;
6
7     int lineNumber = 0;
8     while (current != NULL) {
9         if (current->op != OP_LABEL) {
10            char* opCode = getOpcodeBinary(current->op);
11            char *op1 = NULL, *op2 = NULL, *op3 = NULL, *immed = NULL;
12
13            // Formato 1: R R R 8
14            if (current->addr1.kind == REGISTER_KIND &&
15                current->addr2.kind == REGISTER_KIND &&
16                current->addr3.kind == REGISTER_KIND) {
17
18                op1 = intToBinaryString(current->addr1.val, 6);
19                op2 = intToBinaryString(current->addr2.val, 6);
20                op3 = intToBinaryString(current->addr3.val, 6);
21                immed = intToBinaryString(0, 8);

```

```

22     }
23     // Formatos 2 e 3: R R I14
24     else if (current->addr1.kind == REGISTER_KIND && current->addr2.kind
25             == REGISTER_KIND) {
26         op1 = intToBinaryString(current->addr1.val, 6);
27         op2 = intToBinaryString(current->addr2.val, 6);
28
29         int val = 0;
30         if (current->addr3.kind == LABEL_KIND) {
31             val = getLineByLabel(labelHead, current->addr3.name);
32             // Regra específica para Branch: (label - (PC + 1))
33             if (isBranch(current->op)) {
34                 val -= (lineNumber + 1);
35             }
36         } else {
37             val = current->addr3.val;
38         }
39         immed = intToBinaryString(val, 14);
40     }
41     // Formatos 4, 5 e 6: R I20
42     else if (current->addr1.kind == REGISTER_KIND) {
43         op1 = intToBinaryString(current->addr1.val, 6);
44         int val = (current->addr2.kind == LABEL_KIND) ? getLineByLabel(
45             labelHead, current->addr2.name) : current->addr2.val;
46         immed = intToBinaryString(val, 20);
47     }
48     // Formato 7: I26
49     else {
50         int val = (current->addr1.kind == LABEL_KIND) ? getLineByLabel(
51             labelHead, current->addr1.name) : current->addr1.val;
52         immed = intToBinaryString(val, 26);
53     }
54
55     fprintf(file, "%s", opCode);
56     if (op1) fprintf(file, "%s", op1);
57     if (op2) fprintf(file, "%s", op2);
58     if (op3) fprintf(file, "%s", op3);
59     if (immed) fprintf(file, "%s", immed);
60     fprintf(file, "\n");

```

Código 5.14 – Montagem das instruções binárias (binary.c)

5.2.5 Gerenciamento de Memória

Como o processador tem uma quantidade pequena de registradores, boa parte das variáveis e dos parâmetros das funções precisa ser guardada na memória. Para organizar esse uso, o gerenciamento de memória foi feito com base em duas pilhas, que crescem em sentido decrescente a partir do topo da memória.

A memória disponível é definida por uma constante de tamanho igual a 127. A partir desse valor, o registrador R13 funciona como ponteiro da pilha global, onde ficam as variáveis globais, e o registrador R12 funciona como ponteiro da pilha geral, usada durante a execução das funções para guardar variáveis locais, parâmetros e valores temporários.

No início do programa, o ponteiro global é inicializado com o topo da memória, por meio da instrução `movi R13, 127`, e o ponteiro geral é inicializado com o mesmo valor, por meio da instrução `movr R12, R13`.

Para alocar espaço em uma pilha, o ponteiro correspondente é decrementado com a instrução `subi`, e para liberar esse espaço, o ponteiro é incrementado de volta com a instrução `addi`. Dessa forma, cada função aloca o espaço de que precisa ao ser chamada e libera esse espaço ao retornar, o que permite inclusive o funcionamento correto da recursão, já que cada chamada trabalha com a sua própria região da pilha.

6 Desenvolvimento do Sistema operacional

6.1 Gerenciamento de processos

Para o gerenciamento de processos, foi escolhido implementar um escalonador do tipo round-robin, dessa forma, cada processo recebe um determinado número de instruções para serem executadas, e caso não termine nesse tempo, ele é colocado no final da fila de processos prontos, e o próximo processo da fila é executado. O escalonador é acionado a cada 10 instruções executadas, e o processo que estava em execução é colocado no final da fila de processos prontos, e o próximo processo da fila é colocado em execução.

6.2 Gerenciamento de entrada e saída

6.2.1 Interface

Para a interface do sistema operacional com o usuário, será utilizado o display LCD que exibirá mensagens de quais são as escolhas do usuário, bem como qual é o processo que está sendo executado no momento. Para isso, serão passados códigos correspondentes a cada uma das mensagens, além disso, será utilizado um segundo registrador para indicar qual é o processo que está sendo executado no momento.

O controle do display LCD será feito exclusivamente pelo sistema operacional, de modo que nenhum outro programa tenha um acesso à ele. Além disso, para ser implementado contará com uma nova função chamada *showLCD([numero])*. A [Tabela 19](#) abaixo apresenta quais serão os códigos correspondentes a cada uma das mensagens que serão exibidas no display LCD.

Tabela 19 – Mensagens do LCD

Código	Mensagem	Significado
0	0 = um programa; 1 = multiplos	Opções de execução do usuário
1	Digite o numero do programa	Programa a ser executado
2	Digite quantos programas	Nº de programas concorrentes
3	Digite os numeros dos programas	Programas a serem executados
4	Programa [número]	Programa em execução

Fonte: o Autor.

Para isso, será criada uma nova instrução *disp*, similar à instrução *out*, que fará a leitura do registrador de uso específico *R11*, que contém o número do progrma em execução e o enviará para o display LCD junto com a mensagem. Além disso, para que o código

chege ao módulo de entrada e saída, será criado um novo caminho de dados, que fornecerá o valor do imediato da instrução para este módulo.

7 Conclusão

Ao longo deste relatório, foi apresentado o desenvolvimento de um compilador para a linguagem C-, capaz de traduzir um programa de alto nível até o código binário, além do processador MIPS monociclo que serve de alvo para esse código e que foi desenvolvido anteriormente. Com isso, tornou-se possível acompanhar todo o caminho de um programa, desde a sua escrita em C- até a execução das instruções em binário no hardware, passando pela análise léxica, sintática e semântica e, em seguida, pela geração do código intermediário, do assembly e do executável.

7.1 Dificuldades Encontradas

Durante o desenvolvimento do compilador, uma das maiores dificuldades foi a alocação de registradores, já que o processador possui uma quantidade pequena de registradores de uso geral. Foi preciso criar uma forma de reaproveitá-los ao longo do programa e de decidir quando um valor deveria ser guardado na memória para liberar espaço, o que exigiu bastante atenção para não sobrescrever um valor que ainda seria utilizado.

Ligada a essa questão, também foi trabalhosa a situação dos valores temporários que precisam sobreviver a uma chamada de função, pois eles não podem simplesmente ficar em um registrador que a função chamada pode alterar. Para tratar esse caso, foi necessário acompanhar o uso de cada temporário e proteger aqueles que atravessam uma chamada.

Outro ponto que gerou dificuldade foi o gerenciamento de memória com as duas pilhas, principalmente para garantir o funcionamento correto da recursão, na qual cada chamada precisa da sua própria região de memória. Além disso, na geração do código binário, foi preciso ter cuidado no cálculo do destino dos desvios, que é feito de forma relativa em relação à próxima instrução, e na conversão das quádruplas genéricas para o formato de instrução correto de acordo com o tipo dos operandos.

Em relação ao processador, durante o seu desenvolvimento, foram encontradas algumas contradições entre a operação realizada pelas instruções `storer` e `storei` e o caminho de dados. Além disso, o salto condicional a princípio não estava considerando a posição atual do PC. Ambos os erros já foram corrigidos.

Ainda sobre o processador, inicialmente estava planejado executar as operações de multiplicação e divisão utilizando dois registradores especiais para armazenar o resultado, RH, para a parte alta da multiplicação e RL, para a parte baixa da multiplicação. Porém, constatou-se que isso iria fazer a utilização do processador ser consideravelmente mais

complicada. Isso levou a uma mudança para permitir que a parte baixa do resultado pudesse ser escrita em qualquer registrador. Foi também abandonada a ideia de utilizar o registrador específico RH, pois, durante o desenvolvimento da integração entre a ULA e o banco de registradores, foi observado que o tempo de compilação aumentava de cerca de 11 minutos para meia hora ao utilizar duas conexões entre a ULA e o banco de registradores. Sendo assim, o projeto passou a utilizar somente a saída padrão, SL, para essas instruções.

Além disso, para receber e enviar dados para o módulo de entrada e saída, a princípio, seriam utilizados exclusivamente dois registradores especiais, RIN, para receber os dados, e ROUT para enviar. Todavia, isso também poderia dificultar a utilização do processador futuramente, então passou a ser permitido que qualquer registrador desempenhe essas funções. Um outro desafio encontrado foi o caminho de dados, que originalmente havia sido desenvolvido de um modo muito ineficiente. Desse modo, foi preciso corrigi-lo, tanto para ser mais fácil de compreendê-lo quanto para ser mais simples de implementá-lo. Por fim, na etapa de testes no kit FPGA, foram encontradas grandes dificuldades para escrever corretamente os valores no display, já que o valor era escrito em um ciclo de clock e perdido no ciclo seguinte. Após muitos testes, a solução encontrada foi escrever no display somente durante a borda de descida do clock.

7.2 Destaques

Um dos principais destaques do projeto é o fato de o compilador funcionar de ponta a ponta, ou seja, ele recebe um programa em C- e produz, de forma automática, o código intermediário, o código assembly e o código binário correspondente. Além disso, o binário gerado utiliza exatamente os mesmos opcodes e formatos do processador desenvolvido anteriormente, o que permite que o código produzido pelo compilador seja de fato executado no hardware implementado no kit FPGA.

Outro ponto de destaque é o suporte à recursão, que pôde ser demonstrado nos exemplos do fatorial e do máximo divisor comum. Poder acompanhar um mesmo programa em todas as suas representações, desde o código-fonte até o binário, ajuda bastante a entender de maneira concreta como um compilador transforma um programa de alto nível em algo que a máquina consegue executar. Em relação ao processador, vale destacar que todos os módulos desenvolvidos foram testados e apresentaram o funcionamento de acordo com o planejado, e a integração entre eles foi feita aos poucos, sem grandes problemas.

7.3 Trabalhos Futuros

No compilador, muitas melhorias ainda poderiam ser feitas. Seria interessante aplicar otimizações ao código gerado, de modo a reduzir a quantidade de instruções produzidas,

além de aprimorar a alocação de registradores e o controle dos valores temporários. Também seria possível ampliar o conjunto de construções da linguagem tratadas pelo compilador e melhorar as mensagens de erro apresentadas ao usuário.

Já no processador, muitas melhorias poderiam ser feitas, incluindo: uma melhor organização do caminho de dados, que poderia ser mais simples e direto; ao invés de escrever em dois registradores ao mesmo tempo no banco, transformar os registradores especiais em buffers que ficariam dentro do módulo onde são usados; teste de mais instruções para garantir ainda mais solidamente o correto funcionamento.

Referências

- AHO, A. V. et al. *Compiladores: princípios, técnicas e ferramentas*. 2. ed. São Paulo: Pearson Addison-Wesley, 2008. ISBN 978-85-88639-24-3. Citado na página 69.
- Free Software Foundation. *Lexical Analysis with Flex*. 2012. <<https://westes.github.io/flex/manual/>>. Acesso em: 20 jun. 2026. Citado na página 61.
- Free Software Foundation. *Bison: The Yacc-compatible Parser Generator*. 2021. <<https://www.gnu.org/software/bison/manual/>>. Acesso em: 20 jun. 2026. Citado na página 61.
- Intel. *Altera® FPGA noções básicas e primeiros passos*. 2024. <<https://www.intel.com.br/content/www/br/pt/support/programmable/support-resources/fpga-training/getting-started.html>>. Acesso em: 18 dez. 2024. Citado na página 13.
- LOUDEN, K. C. *Compiladores: princípios e práticas*. São Paulo: Thomson Learning, 2004. ISBN 85-221-0422-5. Citado na página 61.
- MIDORIKAWA, E. T. *Uma Introdução às Linguagens de Descrição de Hardware*. 2007. <https://www2.pcs.usp.br/~labdig/pdffiles_2011/intro-hdl.pdf>. P. 2–3. Citado na página 13.
- TANENBAUM, A. S.; BOS, H. *Sistemas Operacionais Modernos*. 4. ed. São Paulo: Pearson Education do Brasil, 2016. Tradução: Jorge Ritter. Revisação técnica: Raphael Y. de Camargo. ISBN 978-85-4301-818-8. Citado 5 vezes nas páginas 9, 10, 11, 12 e 13.